Physically-Feasible Reactive Synthesis for Terrain-Adaptive Locomotion

Anonymous Author(s)

***Abstract*—We present an integrated planning framework for quadrupedal locomotion over unforeseen terrain configurations revealed at runtime. Existing methods often depend on heuristics for real-time foothold selection—limiting robustness and adaptability—or rely on computationally intensive trajectory optimization across complex terrains and long horizons. In contrast, our approach combines reactive synthesis for generating correct-by-construction symbolic-level controllers with mixed-integer convex programming (MICP) for physically feasible footstep planning during each symbolic transition. To reduce the reliance on costly MICP solves and accommodate specifications that may be violated due to physical infeasibility, we adopt a symbolic repair mechanism that selectively generates only the required symbolic transitions. During execution, real-time MICP replanning based on actual terrain data, runtime symbolic repair, and a delay-aware coordination scheme between the strategy automaton and the tracking controller enable seamless bridging between offline synthesis and online operation. Through extensive simulation and hardware experiments, we validate the framework's ability to identify missing locomotion skills and respond effectively in safety-critical environments, including scattered stepping stones and rebar scenarios. Code and experimental videos are available at https://synthesis-micp.github.io/.**

***Index Terms*—Legged Robots, Optimization and Optimal Control, Formal Methods in Robotics and Automation**

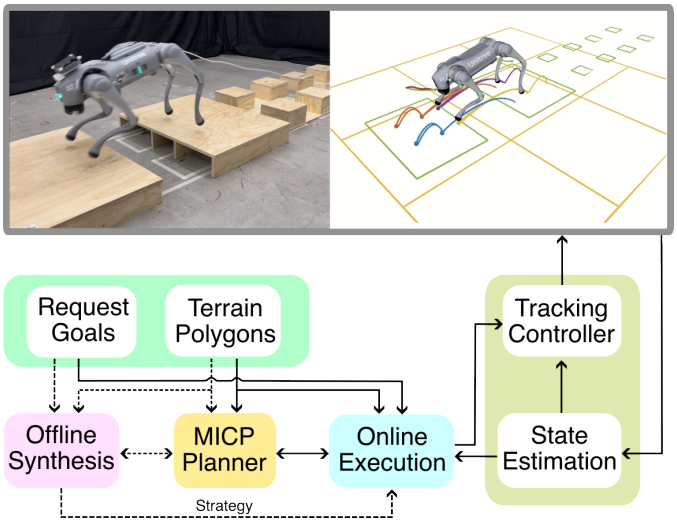


Fig. 1. System architecture overview. The solid lines indicate online communication, while dashed lines represent offline processes. The request goals in the offline phase are user-defined, while those in the online phase are provided by a global planner based on the global goal.

I. Introduction

Terrain-adaptive locomotion is essential for enhancing the mobility of legged robots and moving beyond blind walking strategies di2018dynamic,kim2019highly, zhou2022momentum. Many existing methods achieve terrain adaptability by adjusting footholds around a nominal trajectory in real time fankhauser2018robust,grandia2023perceptive. To further boost traversability, some approaches have explored variable gait patterns. In Park2015OnlinePF, kim2020vision,gilroy2021autonomous, jumping motions are precomputed offline and triggered heuristically upon detecting obstacles. Other methods embed gait switching—such as transitions from walking to jumping—within simplified dynamics models during trajectory sampling norby2020fast, or rely on pre-trained classifiers to assess feasibility chignoli2022rapid. However, despite the importance of safety in high-risk settings such as disaster zones and construction sites, relatively few works address formal safety guarantees for agile, variable-gait locomotion zhao2022reactive,shamsah2023integrated that fully account for the robot's physical limitations.

Mixed-integer programming (MIP)-based methods valenzuela2016mixed,shirai2022simultaneous model both contact state and contact surface selection for each leg and timestep as binary decision variables. This allows them to explicitly capture the interaction between terrain geometry and the robot's kinematic and dynamic constraints deits2014footstep. When dynamics and constraints are suitably relaxed, the resulting convex approximation (MICP) provides a global certificate of feasibility upon convergence dai2019global, making it a powerful tool for formally assessing locomotion viability. However, the computational demands of MIP-based approaches scale poorly with increased terrain complexity and longer planning horizons, posing a major hurdle for real-time applications. To mitigate this, various simplifications have been proposed, such as fixing contact sequences ding2020kinodynamic,fey20243d, constraining timing ponton2016convex,risbourg2022real,acosta2023bipedal, or limiting the number of footsteps aceituno2017simultaneous. Despite these efforts, scalability remains a challenge in cluttered environments with various terrain segments and features.

To simplify navigation across complex terrains, one effective strategy is to decompose long-horizon, global tasks into a sequence of localized, robot-centric environments centered around intermediate waypoints fankhauser2018robust. Despite this simplification, the associated MIP formulation still faces computational challenges, as it must account for all nearby terrain elements. To address this, we discretize the local environment and constrain each MIP to solving only a single discrete transition at a time. These discrete transitions are then composed using a Linear Temporal Logic (LTL)-based reactive synthesis framework bloem2012synthesis, which enables us

to systematically construct a correct-by-construction strategy that guides the robot through local decisions toward achieving its intermediate goals. For readers unfamiliar with formal methods, LTL is a logic for expressing temporal properties of system behaviors using operators such as "always," "eventually," and "until" piterman2006synthesis,bloem2012synthesis; reactive synthesis automatically constructs a controller (a *strategy automaton*) that satisfies an LTL specification against any admissible environment behavior, providing correct-by-construction guarantees at the symbolic level.

In this work, we present an integrated planning framework that unifies reactive synthesis with MIP-based optimization to enable legged robots to respond safely and efficiently to unforeseen terrain configurations revealed at runtime. We abstract both the continuous robot states and local environmental context into symbolic representations, allowing the robot to make high-level decisions in real time for negotiating complex obstacles such as large gaps or cluttered terrain (see Fig. 1). Each symbolic transition is validated through an MICP to ensure the physical feasibility of the locomotion process. This architecture not only connects high-level symbolic reasoning with low-level dynamic feasibility, but also mitigates the computational challenges of solving long-horizon MICP problems. By leveraging guidance from the LTL-based symbolic planner, each MICP instance only needs to handle short horizons and a limited set of terrain features, which substantially reduces computational load. The framework is two-stage: in the offline synthesis stage, we employ *gait-free* MICP formulations that jointly optimize over contact mode and foothold placement to precompute a diverse set of locomotion skills for symbolic transitions. During online deployment, we switch to lightweight *gait-fixed* MICP formulations using these precomputed gaits, enabling real-time generation of physically consistent motions with real-world terrain data.

Our offline synthesis stage tackles two key scalability challenges. First, our task specification has a prohibitively large state space since it needs to encode the surrounding terrain states. Since reactive synthesis has exponential time complexity in the number of variables bloem2012synthesis, it is inefficient and impractical to directly synthesize a controller for the full specification. To tackle this challenge, we leverage the observation that the robot can continuously reason about its local environment where the terrains and an intermediate goal remain unchanged. Thus, we propose a high-level manager that fixes the terrain and goal information, only keeps the skills needed for the current terrains, and updates them as the robot moves. In our experiments, the manager reduces the number of variables by 80.9–90.1% depending on the number of cells and terrain types, relieving the computation burden of synthesis. Second, while the size of the MICP for our physical feasibility certificate is much smaller than the pure MIP approach, solving gait-free MICP is still computationally expensive. To mitigate this problem, we leverage a symbolic repair approach to automatically suggest only the missing, yet necessary, symbolic transitions, and only solve the gait-free MICP for them. In this way, we avoid enumerating and solving the costly MICP for all possible symbolic transitions. In our experiments, symbolic repair reduces the number of expensive MICP solves by 71.6–97.6% depending on the terrains, compared with exhaustively iterating over all possible symbolic transitions, ensuring that we only perform the costly gait-free MICP to generate new locomotion gaits when necessary.

Our online execution stage addresses two complementary challenges. First, discrepancies in size and shape between offline-checked terrains and real-world ones can lead to mismatches between the desired symbolic transitions and the robot's actual physical capabilities. Second, encountering new terrains or intermediate goals not considered in the offline phase may also make symbolic specifications unrealizable, as the unforeseen scenarios are not handled during the offline phase. As such, our online execution module executes each symbolic transition by solving an online gait-fixed MICP with real-world terrain data and prior locomotion gaits to generate a new nominal trajectory. If a symbolic transition is no longer possible or the current terrains and goal were not considered offline, we perform runtime symbolic repair to generate new robot skills and synthesize an updated robot strategy, allowing continuous traversal on unexpected terrains.

The proposed framework remains flexible and modular—it can be driven by commands from a higher-level planner wermelinger2016navigation, fernbach2017kinodynamic,norby2020fast,chignoli2022rapid,liao2023walking,f or directly from a user, and it easily accommodates new behaviors by expanding the symbolic transition set.

The main contributions of this work are summarized as follows:

- **Integrated reactive-synthesis and MICP framework for terrain-adaptive locomotion.** Symbolic-level guidance limits the planning horizon and the number of terrain features any single MICP must consider, bridging high-level symbolic reasoning with low-level physical feasibility.
- **Scalable offline synthesis via a high-level manager and symbolic repair with dynamic feasibility checks.** The high-level manager reduces the symbolic state space by 80.9–90.1%, and symbolic repair, equipped with dynamic feasibility checks via gait-free MICP, generates only the necessary skills—reducing costly gait-free MICP solves by 71.6–97.6% versus exhaustive enumeration.
- **Online execution module that bridges offline synthesis and real-world deployment.** Online gait-fixed MICP with real-world terrain data, paired with runtime symbolic repair, handles both terrain discrepancies and previously unseen scenarios at runtime.
- **Comprehensive empirical validation.** We report simulation across four terrain scenarios with full per-module timing, comparisons against a pure MIP planner and a heuristic foothold planner, hardware deployment of the integrated stack on two robot platforms (Unitree Go2 and SkyMul Chotu), and a preliminary feasibility study of yaw orientation as a generalization direction.

A conference version of this work is presented in zhou2025physically. The current article significantly advances the original paper in several critical aspects. First, we enhance the online execution workflow by addressing inevitable delays

in MICP solving through coordinated scheduling between the strategy automaton and the tracking controller. Second, we introduce online retargeting of the robot's desired pose at each transition to ensure feasibility under the actual terrain configuration. Finally, we validate the full framework through hardware experiments and benchmark its performance against both a heuristics-based footstep planner and a pure MIP planner. We also include an extension study incorporating yaw orientation to further demonstrate the generalizability of the proposed framework.

II. Related Work

A. Hierarchical Planning for Terrain-Adaptive Locomotion

Planning for terrain-adaptive locomotion can be boiled down into solving contact sequence and timing (equivalently gait), location (equivalently foothold), and robot motion (e.g., Center of Mass (CoM) or base pose), given the chosen kinematics/dynamics model and the perceived terrain information. Hierarchical approaches first separately or simultaneously address part of the above problems and then solve for the rest, which allows for higher computational efficiency. Kinematic footstep planning focuses on the first two problems, neglecting the robot dynamics through graph-based approaches kolter2008control,kalakrishnan2010fast,winkler2014path,mastalli2015line,fankhauser2018robust,griffin2019footstep or optimization-based approaches deits2014footstep,tonneau2020sl1m,risbourg2022real, ren2025accelerating. Although non-fixed gait/contact plans can be generated for very rough terrains hauser2005non,bretl2006motion,tonneau2018efficient, conservative quasi-static motions are generated for challenging terrains due to the kinematic simplification during footstep planning.

Given the recent advances in optimization-based approaches, notably Model Predictive Control (MPC), another focus of the literature is on the local foothold adaptation and the integration of robot dynamics for dynamic locomotion. Instantaneous foothold adaptation around a nominal location, with a fixed and cyclic gait, is performed based on heuristic search jenelten2020perceptive,kim2020vision,agrawal2022vision or learning approaches magana2019fast, villarreal2020mpc,yu2021visual,gangapurwala2022rloc, wu2025learn, Kamohara2025RLMPC. In grandia2021multi,jenelten2022tamols,grandia2023perceptive, the base pose and foothold are optimized jointly, demonstrating impressive terrain traversing capabilities on rough terrains. More recently, MPC has been employed within hierarchically integrated planning frameworks for legged navigation over rough terrain, incorporating explicit terrain uncertainty quantification muenprasitivej2024bipedal, shamsah2025terrain, muenprasitivej2026bipedal.

However, the above dynamic locomotion works consider a fixed gait pattern that usually prohibits achieving versatile behaviors, such as jumping. In Park2015OnlinePF, kim2020vision,gilroy2021autonomous, jumping motions are generated offline and triggered through heuristics. In fernbach2017kinodynamic,norby2020fast,chignoli2022rapid, RRT-Connect is used for rapid kino-dynamic locomotion planning, and the switch between normal walking and jumping is either embedded inside a reduced-order model during sampling norby2020fast or decided by a pre-trained feasibility classifier chignoli2022rapid. Beyond limited gait modes, other works focus on online gait planning and/or foothold selection and deploy MPC to track the plan, which can be further categorized into model-free da2021learning,yang2022fast,xie2022glide,pmlr-v164-margolis22a,yu2024learning and model-based methods boussema2019online,wang2023and,sun2023free.

A complementary line of work uses end-to-end reinforcement learning to train perceptive locomotion policies that directly map terrain observations to joint commands, achieving impressive robustness on stairs, gaps, and unstructured outdoor terrain hoeller2023anymal,gangapurwala2022rloc,miki2022elevation,pmlr-v164-margolis22a. Compared with these data-driven approaches, our model-based, formal-methods framework offers explicit symbolic-level realizability guarantees and physical-feasibility certificates from MICP, at the cost of weaker generalization and the need for a terrain abstraction; we view the two approaches as complementary rather than competing.

B. Simultaneous Planning via Optimization

Planning for terrain-adaptive locomotion problem can also be formulated as a combinatorial optimization problem that simultaneously optimizes for gait, foothold, and robot dynamics. One approach is to incorporate rigid posa2014direct or smoothed mordatch2012discovery, drnach2021robust complementarity constraints, which accurately model the physical contact interaction behavior but remains computationally inefficient due to the non-smoothness and high nonlinearity. Promising online contact-implicit MPC results are reported in aydinoglu2022real,le2024fast, drnach2022mediating but still limited to low-dimensional system or static environment.

Another way of encoding discrete contact decisions is to treat both the contact state and the contact plane selection for each leg at each timestep as binary variables valenzuela2016mixed,shirai2022simultaneous,jiang2023locomotion. The underlying problem can be formulated as Mixed Integer Program (MIP). Convex approximations of nonlinear dynamics, usually centroidal dynamics, and constraints are typically required to transform the MIP into a more tractable Mixed Integer Convex Program (MICP), albeit with an increased number of binary variables. A global certificate for the approximated convex problem exists upon convergence dai2019global, providing an ideal means to determine the feasibility in our proposed method. To relieve the computational burden due to the introduction of many binary variables, the works of ponton2016convex,ding2020kinodynamic,acosta2023bipedal assume the contact sequence and timing are chosen *a priori*, and only optimize the contact plane selection. Aceituno-Cabezas et al. aceituno2017simultaneous optimize the contact

sequence within a certain gait cycle that fixes the number of footsteps. However, it is inevitable that the problem complexity grows exponentially when the number of discrete contact options increases along with the time horizon and terrain features. Our work aims to alleviate computational burden through a two-stage approach. In the offline phase, expensive MICPs that optimize both contact state and foothold selection are solved to generate necessary locomotion gaits. In the online phase, efficient MICPs with adaptive and predefined gaits are used to produce dynamically feasible motions and footholds. Additionally, guided by a synthesis-based task planner, each MICP in our work considers shorter time horizons and fewer terrain features compared to solving a single large MICP problem.

C. Task and Motion Planning for Contact-Rich Planning

Task and Motion Planning (TAMP) formally defines symbolic-level tasks and searches through a graph of predefined motion primitives that enable feasible symbolic transitions garrett2021integrated, zhao2024survey. For more complex physical contact resoning, Toussaint et al. propose Logic Geometric Programming (LGP) toussaint2015logic and embed the high-level logic representation into the low-level motion planner, demonstrating contact-rich tool-use behaviors toussaint2018differentiable after defining abundant action primitives. Building on this, a broader range of motions has been showcased, including versatile manipulation migimatsu2020object,zhao2021sydebo and loco-manipulation tasks sleiman2023versatile. In a similar vein, works such as ding2021hybrid,shirai2021lto,jelavic2021combined,amatucci2022monte,chen2021trajectotree,zhang2023simultaneous,zhu2023efficient,asse integrate graph-search or sampling-based methods with optimization-based approaches in a holistic manner, abstracting contact modes within a discrete domain. However, the combinatorial nature of these problems often leads to poor scalability due to the explosion of contact modes. Deploying such approaches online in dynamic environments remains an open challenge. From a different perspective, we abstract a smaller set of task-level contact modes as locomotion gaits over a specific time horizon and distance, allowing the MICP to solve for details such as contact locations and robot motion during execution. This shifts the focus to selecting locomotion gaits within a local environment while ensuring safety and completeness through synthesis-based approaches.

Unlike LGP that solves an integrated TAMP problem, synthesis-based approaches using Linear Temporal Logic (LTL) operate primarily at the task level, emphasizing safety and completeness guarantees within the task domain kress2018synthesis. These methods typically synthesize a sequence of task-level actions, which are then executed by a motion planner. Recently, LTL has been applied to safe locomotion tasks, employing distinct task-level abstractions on topological maps of terrain kulgod2020temporal,zhou2022reactive,jiang2023abstraction or locomotion keyframes for reduced-order models gu2022reactive,shamsah2023integrated,zhao2022reactive. Reactive synthesis pnueli1989synthesis, widely applied to mobile robots kress2009temporal, has been employed to enable prompt decision-making in response to more complex environments shamsah2023integrated or external perturbations gu2022reactive. Notably, zhao2022reactive formulate the terrain-adaptive locomotion problem as a sequential decision-making task, selecting appropriate locomotion modes with predefined contact and locomotion keyframes to accommodate dynamically changing terrains. However, the locomotion mode selection is explicitly encoded in the LTL specification using expert knowledge, lacking a physically feasibility guarantee. In this work, we aim to combine the strengths of reactive synthesis and MICP, utilizing the global certificate of MICP as a feasibility checker for terrain-adaptive locomotion.

D. Physically-Feasible Reactive Synthesis

While reactive synthesis can promptly and safely respond to dynamic environments, it typically does not assess physical feasibility when being deployed on complex robotic systems. To bridge the gap between discrete abstraction and continuous system dynamics, various strategies have been proposed. Studies in decastro2014dynamics,fainekos2006translating focus on automatically synthesizing controllers in the continuous domain based on high-level specifications. Another approach involves incorporating dynamics directly into the reactive synthesis process by abstracting physical systems, including nonlinear bhatia2010sampling, switched liu2013synthesis, and hybrid systems maly2013iterative with manageable model complexity. However, for legged robots with multiple contacts navigating dynamic terrains, these physically feasible reactive synthesis approaches remain computationally intractable.

Recent efforts pacheck2020finding, pacheck2022physically focus on symbolic repair for unrealizable task specifications, defining symbolic skills with preconditions and postconditions, similar to TAMP, to identify missing skills that make the specification realizable. These works introduce iterative feasibility checks based on symbolic repair suggestions, emphasizing alignment between symbolic task specifications and physical reality. Building upon this, Meng et al. meng2024automated extend this type of approach to online symbolic repair. Inspired by these works, Zhou et al. zhou2025physically adopt a similar mechanism for terrain-adaptive locomotion. Specifically, they abstract the robot and terrain states in a local environment and define physically feasible skills by solving MICP. They leverage high-level manager and symbolic repair to efficiently identify missing but necessary skills, reducing the need for frequent calls to the computationally expensive gait-free MICP. Our work significantly completes zhou2025physically through seamless integration with a low-level tracking control module, enabling systematic benchmarking, and demonstrating real-world hardware deployment.

III. Preliminaries

Consider a robot equipped with sensors navigating an environment toward a designated global goal $g_{\text{global}} \in \mathbb{R}^2$. This task can be broken into a sequence of local navigation subtasks, where the robot moves toward intermediate local request goals $\mathcal{G}_{\text{local}}$, determined by a global planner based on the nearby

segmented terrain polygons $\mathcal{P}$. We illustrate an instance of such a local task setup, following the same example introduced in zhou2025physically.

Example 1. *Consider a 2D grid world with nine cells (in yellow) in Fig. 2. The robot is required to move from an initial cell (e.g. the center cell) to a desired goal cell indicated by the star. Each cell is assigned a terrain type, such as Dense or Sparse stepping stones.*

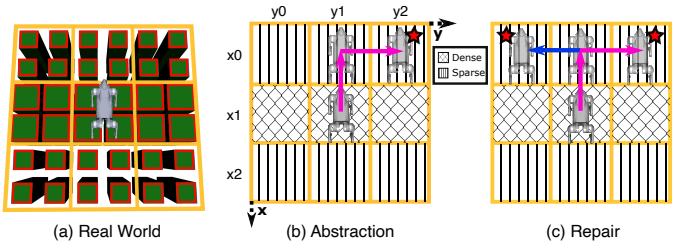


Fig. 2. Terrain abstraction and skill definition. (a) Top-down view of the real-world terrain before abstraction. The red polygons denote the segmented terrain polygons. (b) Abstraction of the terrain and robot's skills (pink) moving from one location to another. (c) Repair process to find a new skill (blue).

A. Abstractions

Given a set of terrain polygons $\mathcal{P}$, such as those in Fig. 2(a), we abstract them into a 2D grid of dimension $n \times m$. For each axis, we label every position with a symbol, producing two sets: $\mathcal{X} := \{x_0, \ldots, x_{n-1}\}$ and $\mathcal{Y} := \{y_0, \ldots, y_{m-1}\}$.

We introduce a collection of atomic propositions AP, partitioned into input variables $\mathcal{I}$ and output variables $\mathcal{O}$, which capture the symbolic world state and robot actions. The inputs $\mathcal{I}$ include robot position inputs $\mathcal{I}_{\text{robot}}$, request inputs $\mathcal{I}_{\text{req}}$, and terrain inputs $\mathcal{I}_{\text{terrain}}$, with $\mathcal{I} = \mathcal{I}_{\text{robot}} \cup \mathcal{I}_{\text{req}} \cup \mathcal{I}_{\text{terrain}}$. The robot inputs are defined as $\mathcal{I}_{\text{robot}} := \{\pi_x \mid x \in \mathcal{X}\} \cup \{\pi_y \mid y \in \mathcal{Y}\}$ to represent the robot's location. Request inputs are given by $\mathcal{I}_{\text{req}} := \{\pi_x^{\text{req}} \mid x \in \mathcal{X}\} \cup \{\pi_y^{\text{req}} \mid y \in \mathcal{Y}\}$ to represent the desired goal cell. The terrain inputs are defined as $\mathcal{I}_{\text{terrain}} := \{n_x^y \mid x \in \mathcal{X}, y \in \mathcal{Y}\}$, which describe the terrain type associated with each grid cell. We assume a finite set of terrain types of size n_t, defined according to their physical characteristics. For $z \in \mathbb{N}$, we denote $[z] := \{0, \ldots, z-1\}$. Accordingly, each terrain input $n_x^y \in \mathcal{I}_{\text{terrain}}$ is an integer variable taking values from $[n_t]$, which we further translate into Boolean propositions following the approach of ehlers2016slugs. The robot state abstraction can also be extended to incorporate orientation, such as the yaw angle of the floating base (see Sec. VIII-H).

An input state $\sigma_{\mathcal{I}}$ is a mapping $\sigma_{\mathcal{I}} : \mathcal{I} \to \{\texttt{True}, \texttt{False}\} \cup [n_t]$. Because the robot and its goal occupy only a single grid cell each, exactly one π_x, one π_y, one π_x^{req}, and one π_y^{req} must evaluate to `True` for any valid input state. We denote projections of $\sigma_{\mathcal{I}}$ as robot states $\sigma_{\text{robot}} : \mathcal{I}_{\text{robot}} \to \{\texttt{True}, \texttt{False}\}$, request states $\sigma_{\text{req}} : \mathcal{I}_{\text{req}} \to \{\texttt{True}, \texttt{False}\}$, and terrain states $\sigma_{\text{terrain}} : \mathcal{I}_{\text{terrain}} \to [n_t]$. The complete sets of such states are denoted by $\Sigma_{\mathcal{I}}$, Σ_{robot}, Σ_{req}, and Σ_{terrain}, respectively.

We introduce a grounding function to associate symbolic input states with their physical interpretations. Let the physical state space be $\mathcal{Z} \subseteq \mathbb{R}^n$, which encodes the physical world, including the robot's position, orientation, and terrain attributes. The grounding function $\text{G} : \Sigma_{\mathcal{I}} \to 2^{\mathcal{Z}}$ maps each input state $\sigma_{\mathcal{I}}$ to the corresponding set of physical states in $\mathcal{Z}$.

For robot states $\sigma_{\text{robot}} \in \Sigma_{\text{robot}}$, we define $\text{G}(\sigma_{\text{robot}}) := \{z \in \mathcal{Z} \mid \exists \pi_x, \pi_y \in \mathcal{I}_{\text{robot}}. \sigma_{\text{robot}}(\pi_x) \wedge \sigma_{\text{robot}}(\pi_y) \wedge \text{robot\_pose}(z) \in \text{cell}(x, y)\}$. Similarly, for request states $\sigma_{\text{req}} \in \Sigma_{\text{req}}$, we set $\text{G}(\sigma_{\text{req}}) := \{z \in \mathcal{Z} \mid \exists \pi_x^{\text{req}}, \pi_y^{\text{req}} \in \mathcal{I}_{\text{req}}. \sigma_{\text{req}}(\pi_x^{\text{req}}) \wedge \sigma_{\text{req}}(\pi_y^{\text{req}}) \wedge \text{request\_goal}(z) \in \text{cell}(x, y)\}$. For terrain states $\sigma_{\text{terrain}} \in \Sigma_{\text{terrain}}$, the grounding $\text{G}(\sigma_{\text{terrain}})$ is defined as the set of physical states whose terrain abstractions are consistent with σ_{terrain}. Given an input state $\sigma_{\mathcal{I}} \in \Sigma_{\mathcal{I}}$ with projections σ_{robot}, σ_{req}, and σ_{terrain}, we combine them as $\text{G}(\sigma_{\mathcal{I}}) := \text{G}(\sigma_{\text{robot}}) \cap \text{G}(\sigma_{\text{req}}) \cap \text{G}(\sigma_{\text{terrain}})$. Finally, we introduce an inverse grounding function $\text{G}^{-1} : \mathcal{Z} \to \Sigma_{\mathcal{I}}$ that maps each physical state $z \in \mathcal{Z}$ back to its corresponding symbolic input state.

In Example 1, the inputs are defined as $\mathcal{I}_{\text{robot}} := \{\pi_{x_0}, \pi_{x_1}, \pi_{x_2}, \pi_{y_0}, \pi_{y_1}, \pi_{y_2}\}$, $\mathcal{I}_{\text{req}} := \{\pi^{\text{req}} \mid \pi \in \mathcal{I}_{\text{robot}}\}$, and $\mathcal{I}_{\text{terrain}} := \{n_{x_i}^{y_j} \mid i, j \in 0, 1, 2\}$, where $n_{x_i}^{y_j} = 1$ if the grid cell (x_i, y_j) is classified as Dense, and $n_{x_i}^{y_j} = 0$ if it is Sparse. The input state shown in Fig. 2(b) is $\sigma_{\mathcal{I}} : \pi_{x_1} \mapsto \texttt{True}, \pi_{y_1} \mapsto \texttt{True}, \pi_{x_0}^{\text{req}} \mapsto \texttt{True}, \pi_{y_2}^{\text{req}} \mapsto \texttt{True}, n_{x_1}^{y_j} \mapsto 1$ for $j \in \{0, 1, 2\}$. For brevity, throughout this paper we omit Boolean propositions set to `False` and integer propositions with value 0 when describing input states.

The output propositions $\mathcal{O}$ represent skills that enable the robot to move between grid cells under specific terrain conditions. A skill $o \in \mathcal{O}$ is defined by its preconditions $\Sigma_o^{\text{pre}} \subseteq \Sigma_{\text{robot}} \times \Sigma_{\text{terrain}}$, which specify the robot and terrain states from which it can be executed, and its postconditions $\Sigma_o^{\text{post}} \subseteq \Sigma_{\text{robot}}$, which describe the resulting robot state after execution. In Example 1, the skill o_0 moves the robot from the middle cell to the upper cell (Fig. 2b). The precondition of o_0 is $\Sigma_{o_0}^{\text{pre}} = \{(\sigma_{\text{robot}}, \sigma_{\text{terrain}}) : \pi_{x_1} \mapsto \texttt{True}, \pi_{y_1} \mapsto \texttt{True}, n_{x_1}^{y_0} \mapsto 1, n_{x_1}^{y_1} \mapsto 1, n_{x_1}^{y_2} \mapsto 1\}$, while the postcondition is $\Sigma_{o_0}^{\text{post}} = \{\sigma_{\text{robot}} : \pi_{x_0} \mapsto \texttt{True}, \pi_{y_1} \mapsto \texttt{True}\}$. Each symbolic skill is also paired with a continuous-level locomotion gait, such as a one-second trotting gait L, which physically realizes the symbolic transition (Sec. V-A).

B. Specifications

We adopt the Generalized Reactivity (1) (GR(1)) fragment of linear temporal logic (LTL) bloem2012synthesis to encode task specifications. An LTL formula φ follows the grammar $\varphi := \pi \mid \neg\varphi \mid \varphi \wedge \varphi \mid \bigcirc\varphi \mid \square\varphi \mid \lozenge\varphi$, where $\pi \in AP$ denotes an atomic proposition, $\neg$ and $\wedge$ are Boolean operators, and $\bigcirc$ ("next"), $\square$ ("always"), and $\lozenge$ ("eventually") are temporal operators. For a comprehensive introduction to LTL, we refer the reader to baier2008principles.

A GR(1) specification is written in the form $\varphi = \varphi_{\text{e}} \to \varphi_{\text{s}}$, where $\varphi_{\text{e}} = \varphi_{\text{e}}^{\text{i}} \wedge \varphi_{\text{e}}^{\text{t}} \wedge \varphi_{\text{e}}^{\text{g}}$ encodes the assumptions about the (possibly adversarial) environment, and $\varphi_{\text{s}} = \varphi_{\text{s}}^{\text{i}} \wedge \varphi_{\text{s}}^{\text{t}} \wedge \varphi_{\text{s}}^{\text{g}}$ represents the guarantees that define the system's behavior. For $\alpha \in \{e, s\}$, the components $\varphi_\alpha^{\text{i}}, \varphi_\alpha^{\text{t}}, \varphi_\alpha^{\text{g}}$ describe the initial conditions, safety requirements, and liveness objectives, respectively. We refer below and throughout the rest of this article to *symbolic repair*, the process of augmenting the symbolic transition set with new, dynamically feasible skills whenever the current specification becomes unrealizable. The

full procedure is formalized in Sec. V-C; for the purpose of the upcoming definitions, it suffices to treat symbolic repair as a black box that proposes candidate symbolic transitions whose physical feasibility is then verified by MICP. In the context of symbolic repair (Sec. V-C), we further partition the safety constraints $(\varphi_e^t, \varphi_s^t)$ into two parts: skill-related constraints $(\varphi_e^{t,skill}, \varphi_s^{t,skill})$ and hard constraints $(\varphi_e^{t,hard}, \varphi_s^{t,hard})$. The skill constraints encode the preconditions and postconditions of skills (see Sec. V-B) and are subject to modification by the repair procedure, whereas the hard constraints remain immutable.

In practice, GR(1) specifications often become quite large because they must capture detailed information about both terrain and task constraints. At runtime, however, the exact terrain and request states are already available. This allows us to apply a technique known as partial evaluation originally defined in zhou2025physically, where known Boolean propositions are directly substituted with their truth values. By doing so, we reduce the overall state space, leading to a smaller and more computationally efficient specification for synthesis.

Definition 1. Given an LTL formula φ and two subsets $S^{\texttt{True}}, S^{\texttt{False}} \subseteq AP$, $S^{\texttt{True}} \cap S^{\texttt{False}} = \emptyset$, we define the *partial evaluation* of φ over $S^{\texttt{True}}, S^{\texttt{False}}$, as $\varphi[S^{\texttt{True}}, S^{\texttt{False}}]$, where we substitute propositions $\pi \in AP$ in φ with `True` if $\pi \in S^{\texttt{True}}$ and with `False` if $\pi \in S^{\texttt{False}}$.

IV. Problem Statement and Approach Overview

A. Problem Statement

Problem 1. Given (i) a global goal g_{global}, (ii) a set of possible local request goals $\mathcal{G}_{\text{local}}$ (iii) a set of predefined terrain polygons $\mathcal{P}_{\text{pre}}$ from user's prior knowledge, and (iv) a set of online terrain polygons $\mathcal{P}_{\text{on}}$ perceived during execution that may differ from the predefined ones, (v) a set of predefined locomotion gaits $\mathcal{L}$; generate controls that enable the robot to navigate the environment and reach the goal.

B. Approach Overview

To address Problem 1 efficiently, we manage complexity on both symbolic and physical levels. At the symbolic level, we employ reactive synthesis to break down the local navigation task into smaller subproblems, which can be reused across different scenarios. Each subproblem involves planning a short-horizon symbolic transition and is accompanied by solving a MICP to certify its physical feasibility.

As shown in Fig. 1, our overall framework is composed of three main modules: offline synthesis (Sec. V), online execution (Sec. VI), and low-level tracking control (Sec. VII). During offline synthesis, we generate a diverse set of locomotion skills and their associated gaits for various predefined terrain-task combinations. Symbolic repair is applied to discover additional necessary transitions that may not be captured initially.

At runtime, we invoke a symbolic repair mechanism to synthesize new transitions when the robot encounters unexpected terrain conditions or request goals not seen during offline planning. This allows the framework to remain flexible and adaptive to evolving environments. Additionally, the system re-targets the desired pose during each transition to reflect current terrain constraints, and coordinates the strategy automaton with the tracking controller to handle MICP solve-time variability without compromising execution continuity.

V. Offline Synthesis

The offline synthesis module builds a strategy that allows the robot to reach local request goals under predefined terrain conditions. As shown in Fig. 3(a), the module takes in local request goals $\mathcal{G}_{\text{local}}$, terrain polygons $\mathcal{P}_{\text{pre}}$ from prior maps, and a set of locomotion gaits $\mathcal{L}$ specified by the user. We first apply the inverse grounding function G^{-1} to discretize the local environment, deriving a set of candidate request states $\Sigma_{\text{req}}^{\text{poss}} \subseteq \Sigma_{\text{req}}$ from the local request goals and mapping the predefined terrain polygons into possible terrain states $\Sigma_{\text{terrain}}^{\text{poss}} \subseteq \Sigma_{\text{terrain}}$. Afterward, we evaluate the feasibility of each candidate skill using a gait-fixed MICP formulated with the provided locomotion gaits (Sec. V-A), and record all feasible transitions as symbolic skills in the specification (Sec. V-B). To improve efficiency, a high-level manager then generates partial evaluations of the encoded specifications. Whenever one of these partial evaluations is deemed unrealizable, a repair mechanism is invoked: it proposes additional symbolic skills, whose feasibility is validated through gait-free MICP. This process ensures that the specification ultimately becomes realizable (Sec. V-C).

A. Locomotion Gait and Feasibility Checking via MICP

We assume each locomotion gait L is characterized by a contact sequence $\mathcal{G}$ and its associated time durations T, formally written as $L = \mathcal{M}(\mathcal{G}, T)$. Each skill is tied to a distinct locomotion gait, which dictates how the robot executes motion at the continuous level. As illustrated in Fig. 3(a), since the offline phase has access to the complete set of request states $\Sigma_{\text{req}}^{\text{poss}}$ and terrain states $\Sigma_{\text{terrain}}^{\text{poss}}$, we can systematically enumerate all possible symbolic skills that enable navigation across the different local environments. The feasibility of each skill is then evaluated through gait-fixed MICP using the given locomotion gaits. In most examples shown in this paper, we assume the robot is only allowed to move horizontally and vertically by one cell and we show diagonal movements and heading angle change in Sec. VIII-H.

Then the next critical step is to find whether there exists a locomotion gait for each skill to be physically feasible. We leverage mixed-integer convex programming (MICP) to check the physical feasibility given a set of predefined locomotion gaits, and only use the feasible one as robot skills to synthesize robot strategies. The MICP problem takes in the centroidal states $\mathcal{I}_{\text{robot}}$ from the precondition and postcondition of the skill as its initial and final conditions, which are located at the center of each abstracted cell. A set of homogeneous, predefined terrain polygons are considered in the MICP as steppable regions corresponding to the terrain states $\mathcal{I}_{\text{terrain}}$ involved in the precondition. Lastly, the contact information $\mathcal{G}$ and T is provided by the selected locomotion gait L. Fig. 4 shows the maneuver of the quadruped robot Go2 after solving the MICP problem corresponding to skill a_0 in Example 1

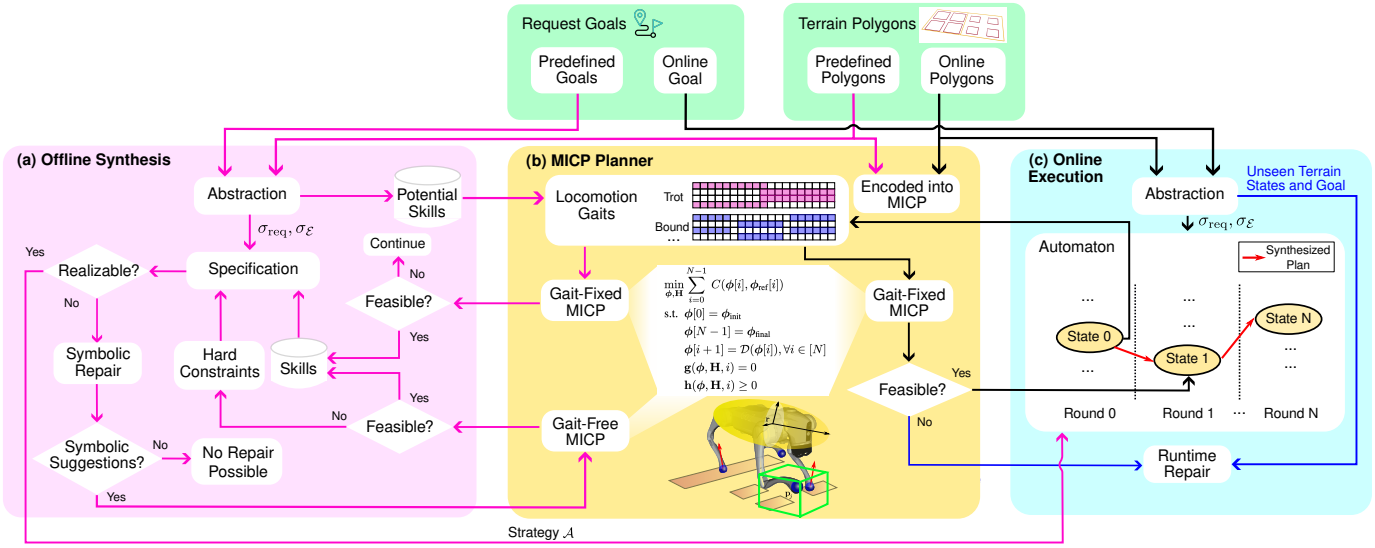


Fig. 3. System overview. During offline synthesis (pink arrows), an initial set of locomotion gaits is provided, and symbolic skills are iteratively generated by solving the MICP. When the task specifications are unrealizable, a symbolic repair is triggered to seek missing skills. During the online execution (black arrows), MICP is solved again taking online terrain segments and the symbolic state only advances when a solution is found. A runtime repair (blue arrows) is initiated if a solving failure occurs, or an unseen terrain or request goal is encountered.

with a one-second trotting locomotion gait. The generic MICP problem considered in this work can formulated as:

$$\min_{\boldsymbol{\phi},\mathbf{H}} \sum_{i=0}^{N-1} C(\boldsymbol{\phi}[i], \boldsymbol{\phi}_{\text{ref}}[i])$$
$$\text{s.t. } \boldsymbol{\phi}[0] = \boldsymbol{\phi}_{\text{init}} \tag{1a}$$
$$\boldsymbol{\phi}[N-1] = \boldsymbol{\phi}_{\text{final}} \tag{1b}$$
$$\boldsymbol{\phi}[i+1] = \mathcal{D}(\boldsymbol{\phi}[i]), \forall i \in [N] \tag{1c}$$
$$\mathbf{g}(\boldsymbol{\phi}, \mathbf{H}, i) = 0 \tag{1d}$$
$$\mathbf{h}(\boldsymbol{\phi}, \mathbf{H}, i) \geq 0 \tag{1e}$$

where the decision variables $\boldsymbol{\phi}$ denote the continuous variables and $\mathbf{H}$ indicate the binary variables across the entire N timesteps. For any natural number $n \in \mathbb{N}$, we denote the set $\{0, \ldots, n-1\}$ as $[n]$. The function $\mathcal{D}(\cdot)$ encodes the discrete-time system dynamics that propagate the continuous state $\boldsymbol{\phi}[i]$ to $\boldsymbol{\phi}[i+1]$; the specific dynamics used in this work are detailed in Eq. (3). The general equality constraints $\mathbf{g}(\cdot)$ and inequality constraints $\mathbf{h}(\cdot)$ are incorporated and activated at certain time stamp i. The cost function depends on both the continuous variables $\boldsymbol{\phi}$ and a reference trajectory $\boldsymbol{\phi}_{\text{ref}}$.

The reference base position and angular trajectories are generated by linearly interpolating between the initial and final conditions. For scenarios with significant elevation changes, such as jumping onto higher terrain, an additional middle keyframe is introduced for the pitch angle to calculate the slope angle between the initial and final poses. The final reference pitch trajectory is then created by evenly interpolating between the initial, middle, and final poses. The reference EE trajectory relative to the base frame ${}^{\mathcal{B}}\mathbf{p}_j^{\text{ref}}$ remains constant and moves in sync with the reference base trajectory.

Given a specific locomotion gait, we first introduce the gait-fixed MICP formulation:

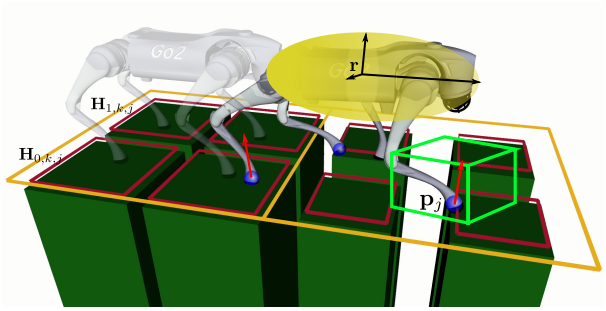


Fig. 4. Demonstration of decision variables and polygons when solving MICP for skill a_0 in Example 1 using a trotting locomotion gait.

1) Continuous Decision Variables: The continuous decision variables $\boldsymbol{\phi}$ include base position $\mathbf{r}$, velocity $\dot{\mathbf{r}}$, acceleration $\ddot{\mathbf{r}}$, orientation $\boldsymbol{\theta}$ parameterized by Euler angle and its first and second-order derivatives $\dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$, individual end-effector (EE) position $\mathbf{p}_j$, velocity $\dot{\mathbf{p}}_j$, and acceleration $\ddot{\mathbf{p}}_j$, and individual contact force $\mathbf{f}_j$ for the foot j with $j \in [n_f]$, where $n_f = 4$ denotes the index of feet. The compact form can be expressed as:

$$\boldsymbol{\phi} = [\mathbf{r}^\top, \dot{\mathbf{r}}^\top, \ddot{\mathbf{r}}^\top, \boldsymbol{\theta}^\top, \dot{\boldsymbol{\theta}}^\top, \ddot{\boldsymbol{\theta}}^\top, \mathbf{p}_j^\top, \dot{\mathbf{p}}_j^\top, \ddot{\mathbf{p}}_j^\top, \mathbf{f}_j^\top]^\top \tag{2}$$

2) System Dynamics: We encode the system dynamics as a simplified single rigid body in Eq. (3), using a double integrator to replace the original Euler equation to keep the dynamics constraint convex. The whole system is discretized through Backward Euler integration with Δt between each time step.

$$\begin{bmatrix} \mathbf{r}[i+1] \\ \dot{\mathbf{r}}[i+1] \\ m\ddot{\mathbf{r}}[i] \\ \boldsymbol{\theta}[i+1] \\ \dot{\boldsymbol{\theta}}[i+1] \end{bmatrix} = \begin{bmatrix} \mathbf{r}[i] + \Delta t \cdot \dot{\mathbf{r}}[i+1] \\ \dot{\mathbf{r}}[i] + \Delta t \cdot \ddot{\mathbf{r}}[i+1] \\ \sum_j \mathbf{f}_j[i] + m\mathbf{g} \\ \boldsymbol{\theta}[i] + \Delta t \cdot \dot{\boldsymbol{\theta}}[i+1] \\ \dot{\boldsymbol{\theta}}[i] + \Delta t \cdot \ddot{\boldsymbol{\theta}}[i+1] \end{bmatrix} \tag{3}$$

where m is the robot mass and g is the gravity term.

We note that the rotational dynamics adopted in Eq. (3) are intentionally simplified: the rotational equation of motion is replaced by a double integrator on the Euler angles, which keeps the constraint convex but *neglects the true angular-momentum dynamics*. This makes the formulation tractable for online deployment, but it limits applicability to highly dynamic motions such as flips, aggressive yaw maneuvers at high speed, or platforms where angular-momentum management is critical (e.g., humanoids). Future work will explore convex relaxations of the full angular-momentum dynamics to expand the range of feasible motions while maintaining online tractability.

3) Cost Function: The cost function consists of tracking costs and regularization terms governed by diagonal matrices $\mathbf{Q}$ and $\mathbf{R}$.

$$\mathbf{C} = \delta\boldsymbol{\phi}_Q[i]^T \mathbf{Q}\, \delta\boldsymbol{\phi}_Q[i] + \boldsymbol{\phi}_R[i]^T \mathbf{R} \boldsymbol{\phi}_R[i] \tag{4}$$

where $\delta\boldsymbol{\phi}_Q$ and $\boldsymbol{\phi}_R$ are defined as:

$$\delta\boldsymbol{\phi}_Q = \begin{bmatrix} \mathbf{r} - \mathbf{r}_{\text{ref}} \\ \boldsymbol{\theta} - \boldsymbol{\theta}_{\text{ref}} \\ \mathbf{p}_j - \mathbf{p}_j^{\text{ref}} \end{bmatrix}, \boldsymbol{\phi}_R = \begin{bmatrix} \ddot{\mathbf{r}} \\ \ddot{\boldsymbol{\theta}} \\ \ddot{\mathbf{p}}_j \\ \mathbf{f}_j \end{bmatrix} \tag{5}$$

Tracking costs include deviation from the desired base and foot EE trajectories. The regularization term includes minimizing the base acceleration, Euler angle acceleration, EE acceleration, and contact forces to encourage motion smoothness. The weights for the tracking terms are defined in Table I.

4) Safe Region Constraint: To incorporate safe region constraints for selecting proper footholds, we introduce binary variables $\mathbf{H}_{r,k,j}$, with r, k, and j expressing the r^{th} convex region, k^{th} footstep, and j^{th} foot and $r \in [R], k \in [n_s]$. One footstep is defined as a full swing phase for a foot. We use n_s and R to represent the number of footsteps specified by the gait configuration and the number of convex terrain polygons to be considered. For example, Fig. 4 shows a case with eight polygons. Eqs. (6) - (7) restrict the robot's EE to stay within one of the convex polygons when the corresponding binary variable $\mathbf{H}_{r,k,j}$ is True. Each polygon is parameterized by an inequality constraint ($\mathbf{A}_r$ and $\mathbf{b}_r$) that defines multiple half-spaces, along with an equality constraint ($\mathbf{A}_{\text{eq},r}$ and $\mathbf{b}_{\text{eq},r}$) that ensures the foothold position lies on a 3D plane. We use $\mathcal{C}_{k,j}$ to denote the set of time steps indicating stance after the k^{th} footstep for the j^{th} foot and the safe region constraint only applies to the first stance time step represented as $\mathcal{C}_{k,j}[0]$. Note that, during the offline phase, the terrain polygons are homogeneous and predefined.

$$\begin{aligned} &\forall r \in [R], k \in [n_s], j \in [n_f] \\ &\mathbf{H}_{r,k,j} \Rightarrow \mathbf{A}_r \mathbf{p}_j[i] \leq \mathbf{b}_r, \forall i \in \mathcal{C}_{k,j}[0] && (6) \\ &\qquad\qquad \mathbf{A}_{\text{eq},r} \mathbf{p}_j[i] = \mathbf{b}_{\text{eq},r}, \forall i \in \mathcal{C}_{k,j}[0] && (7) \\ &\sum_{r=0}^{R-1} \mathbf{H}_{r,k,j} = 1 && (8) \\ &\mathbf{H}_{r,k,j} \in \{0, 1\} && (9) \end{aligned}$$

TABLE I
TRACKING COST WEIGHTS

Cost Term	Weights
$\mathbf{r} - \mathbf{r}_{\text{ref}}$	(1000.0, 1000.0, 1000.0)
$\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{ref}}$	(1000.0, 1000.0, 1000.0)
$\mathbf{p}_j - \mathbf{p}_j^{\text{ref}}$	(1000.0, 1000.0, 1000.0)
$\ddot{\mathbf{r}}$	(10.0, 10.0, 10.0)
$\ddot{\boldsymbol{\theta}}$	(10.0, 10.0, 10.0)
$\ddot{\mathbf{p}}_j$	(0.5, 0.5, 0.5)
$\mathbf{f}_j$	(0.1, 0.1, 0.1)

5) Frictional and Contact Constraints: Frictional constraints are defined in Eqs. (10) - (11), with $\mathbf{n}_r$ and $\mathcal{F}_r$ as the normal vector and friction cone of the r^{th} convex region corresponding to the x and y dimensions of the EE position $\mathbf{p}_j^{xy}$. Contact constraints in Eqs. (12) - (13) forces the EE velocity to be zero during stance phase and the contact force to be zero during the non-contact phase. $\mathcal{C}_j$ represents the set of all time steps indicating stance for the j^{th} foot.

$$\begin{aligned} &\forall r \in [R], k \in [n_s], j \in [n_f] \\ &\mathbf{H}_{r,k,j} \quad \Rightarrow \quad \mathbf{f}_j[i] \cdot \mathbf{n}_r(\mathbf{p}_j^{xy}[i]) \geq 0, \ \forall i \in \mathcal{C}_{k,j} && (10) \\ &\qquad\qquad \mathbf{f}_j[i] \in \mathcal{F}_r(\mu, \mathbf{n}_r, \mathbf{p}_j^{xy}[i]), \ \forall i \in \mathcal{C}_{k,j} && (11) \\ &\dot{\mathbf{p}}_j[i] = 0, \ \forall i \in \mathcal{C}_j && (12) \\ &\mathbf{f}_j[i] = 0, \ \forall i \notin \mathcal{C}_j && (13) \end{aligned}$$

where μ denotes the friction coefficient.

6) Actuation Constraint: Since the joint angles are omitted in the single rigid body model, we use a fixed Jacobian $\mathbf{J}_j(\mathbf{q}_j^{\text{ref}})$ at a nominal joint pose $\mathbf{q}_j^{\text{ref}}$ for each leg to approximately consider the torque limit constraint in Eq. (14). In addition, since the translational and angular motions are decoupled, another actuation constraint on the angular acceleration is also added in Eq. (15).

$$\begin{aligned} &\forall i \in [N], j \in [n_f] \\ &\mathbf{J}_j(\mathbf{q}_j^{\text{ref}})^\top \mathbf{f}_j[i] \leq \boldsymbol{\tau}_{\max} && (14) \\ &\mathbf{I}\ddot{\boldsymbol{\theta}}[i] \leq \boldsymbol{\tau}'_{\max} && (15) \end{aligned}$$

where $\boldsymbol{\tau}_{\max}$ is the joint torque limit, $\boldsymbol{\tau}'_{\max}$ is the torque limit applied on the base, and $\mathbf{I}$ is the moment of inertia for the approximated single rigid body.

7) Kinematics Constraint: Lastly, the kinematics constraint in Eq. (16) strictly limits the possible EE movements to assure safety. Due to the convex nature of this MICP, we can determine the feasibility of a possible symbolic transition by checking if the optimal solution exists.

$$\mathbf{p}_j[i] \in \mathcal{R}_j({}^{\mathcal{B}}\mathbf{p}_j^{\text{ref}}, \mathbf{r}[i], \boldsymbol{\theta}_{\text{ref}}, \mathbf{p}_j^{\max}), \forall i \in [N], j \in [n_f] \tag{16}$$

where $\mathcal{R}_j$ is defined as a 3D box constraint around a nominal foot EE position based on the base position and orientation, and constrained by a maximum deviation $\mathbf{p}_j^{\max}$. Note that the reference orientation $\boldsymbol{\theta}_{\text{ref}}$ is used to avoid nonlinear constraint keep the problem convex.

B. Task Specification Encoding

After identifying a set of feasible robot skills, we encode them into the specification φ as part of the environment

safety assumptions $\varphi_{\mathrm{e}}^{\mathrm{t}}$ and the system safety guarantees $\varphi_{\mathrm{s}}^{\mathrm{t}}$. Each skill is associated with a user-defined precondition set $\Sigma_o^{\mathrm{pre}} \subseteq 2^{\mathcal{I}_{\mathrm{robot}} \cup \mathcal{I}_{\mathrm{terrain}}}$ and postcondition set $\Sigma_o^{\mathrm{post}} \subseteq 2^{\mathcal{I}_{\mathrm{robot}}}$, expressed over the robot and terrain inputs introduced in Sec. III-A. The encoding rules below map these sets into the GR(1) components $\varphi_{\mathrm{e}}^{\mathrm{t,skill}}$ and $\varphi_{\mathrm{s}}^{\mathrm{t,skill}}$ deterministically.

The environment-side skill assumptions $\varphi_{\mathrm{e}}^{\mathrm{t,skill}}$ capture the postconditions of each skill. These assumptions enforce that whenever a skill's precondition holds and the skill is executed, at least one of its associated postconditions must also hold:

$$\varphi_{\mathrm{e}}^{\mathrm{t,skill}} := \bigwedge_{o \in \mathcal{O}} \square((o \wedge \bigvee_{\sigma \in \Sigma_o^{\mathrm{pre}}} \bigwedge_{\pi \in \mathcal{I}_{\mathrm{robot}} \cup \mathcal{I}_{\mathrm{terrain}}} \pi = \sigma(\pi)) \rightarrow \bigvee_{\sigma \in \Sigma_o^{\mathrm{post}}} \bigwedge_{\pi \in \mathcal{I}_{\mathrm{robot}}} \bigcirc(\pi = \sigma(\pi))) \tag{17}$$

In Example 1, the postcondition of skill o_0 is defined as

$$\square(o_0 \wedge \pi_{x_1} \wedge \pi_{y_1} \wedge n_{x_1}^{y_0} = 1 \wedge n_{x_1}^{y_1} = 1 \wedge n_{x_1}^{y_2} = 1 \wedge \ldots \rightarrow \bigcirc \pi_{x_0} \wedge \bigcirc \pi_{y_1} \wedge \neg \ldots) \tag{18}$$

The system-side skill guarantees $\varphi_{\mathrm{s}}^{\mathrm{t,skill}}$ encode the preconditions associated with each skill. These guarantees restrict when a skill may be executed by the robot. Specifically, a skill is permitted to run only if at least one of its preconditions is satisfied:

$$\varphi_{\mathrm{s}}^{\mathrm{t,skill}} := \bigwedge_{o \in \mathcal{O}} \square(\neg(\bigvee_{\sigma \in \Sigma_o^{\mathrm{pre}}} \bigwedge_{\pi \in \mathcal{I}_{\mathrm{robot}} \cup \mathcal{I}_{\mathrm{terrain}}} \bigcirc(\pi = \sigma(\pi))) \rightarrow \neg \bigcirc o) \tag{19}$$

The disjunction $\bigvee_{\sigma \in \Sigma_o^{\mathrm{pre}}}$ in the equation above is taken over precondition pairs in Σ_o^{pre}: a skill becomes eligible to execute when the conjunction of input assignments specified by at least one full pair $(\sigma_{\mathrm{robot}}, \sigma_{\mathrm{terrain}}) \in \Sigma_o^{\mathrm{pre}}$ holds, rather than when only some individual atomic propositions within a pair hold. In Example 1, the precondition of skill o_0 is defined as

$$\square\big(\neg(\bigcirc \pi_{x_1} \wedge \bigcirc \pi_{y_1} \wedge \bigcirc n_{x_1}^{y_0} = 1 \wedge \bigcirc n_{x_1}^{y_1} = 1 \wedge \bigcirc n_{x_1}^{y_2} = 1 \wedge \ldots) \rightarrow \neg \bigcirc o_0\big) \tag{20}$$

The hard constraints $\varphi_{\mathrm{e}}^{\mathrm{t,hard}}$ and $\varphi_{\mathrm{s}}^{\mathrm{t,hard}}$ are constraints that a repair cannot modify (see Sec. V-C). Our system's hard constraints $\varphi_{\mathrm{s}}^{\mathrm{t,hard}}$ only allow the robot to execute one skill at a time: $\square\big(\neg(\bigcirc o \wedge \bigcirc o')\big)$, for any two different skills $o, o' \in \mathcal{O}$. Given that, we encode the following hard assumptions $\varphi_{\mathrm{e}}^{\mathrm{t,hard}}$:

(i) the uncontrollable terrain and request inputs cannot change during execution: $\square(\pi \leftrightarrow \bigcirc \pi)$, $\forall \pi \in \mathcal{I}_{\mathrm{terrain}} \cup \mathcal{I}_{\mathrm{req}}$, and

(ii) the robot inputs $\mathcal{I}_{\mathrm{robot}}$ remain unchanged if no skill is executed: $\square\big(\bigwedge_{o \in \mathcal{O}} \neg o \rightarrow \bigwedge_{\pi \in \mathcal{I}_{\mathrm{robot}}} (\pi \leftrightarrow \bigcirc \pi)\big)$.

C. High-level Manager and Symbolic Repair

We design a high-level manager to efficiently synthesize controllers for encoded specifications under given sets of terrain and request states. The manager takes as input the specification φ, a predefined collection of terrain states $\Sigma_{\mathrm{terrain}}^{\mathrm{poss}} \subseteq \Sigma_{\mathrm{terrain}}$, and a predefined set of request states $\Sigma_{\mathrm{req}}^{\mathrm{poss}} \subseteq \Sigma_{\mathrm{req}}$. For each pair $(\sigma_{\mathrm{terrain}}, \sigma_{\mathrm{req}}) \in \Sigma_{\mathrm{terrain}}^{\mathrm{poss}} \times \Sigma_{\mathrm{req}}^{\mathrm{poss}}$, the manager first converts the integer-valued terrain state $\sigma_{\mathrm{terrain}} : \mathcal{I}_{\mathrm{terrain}} \rightarrow [n_t]$ into a Boolean representation $\sigma_{\mathrm{terrain}}^{\mathcal{B}} : \mathcal{I}_{\mathrm{terrain}}^{\mathcal{B}} \rightarrow \{\texttt{True}, \texttt{False}\}$, following the approach in ehlers2016slugs. We then overload $\sigma_{\mathrm{terrain}}^{\mathcal{B}}$ and σ_{req} to denote the propositions that evaluate to `True`. Using this, the manager generates a partial evaluation $\varphi' := \varphi[\sigma_{\mathrm{terrain}}^{\mathcal{B}} \cup \sigma_{\mathrm{req}}, \mathcal{I}_{\mathrm{terrain}}^{\mathcal{B}} \cup \mathcal{I}_{\mathrm{req}} \setminus \sigma_{\mathrm{terrain}}^{\mathcal{B}} \cup \sigma_{\mathrm{req}}]$ (see Definition 1). Skills whose preconditions are evaluated to `False` under the partial evaluation are discarded, which reduces the specification to robot inputs $\mathcal{I}_{\mathrm{robot}}$ and a smaller skill set as outputs. This step prevents exponential growth of the synthesis procedure in the variable size. Finally, the manager synthesizes a strategy $\mathcal{A}_{\varphi'}$ for each reduced specification.

If φ' is unrealizable, this indicates that gait-fixed MICP feasibility checks (Sec. V-A) failed to produce sufficient skills to reach the request under the given terrain. To address this, we rely on gait-free MICP (Sec. V-D), which relaxes the fixed contact sequence and timing constraints while retaining the continuous dynamics. Since gait-free MICP is computationally demanding, we employ symbolic repair pacheck2022physically to generate targeted symbolic suggestions that minimize unnecessary solves. Symbolic repair works by systematically modifying pre- and postconditions of existing skills to propose new candidates. These suggestions are then validated by gait-free MICP; if feasible, they are incorporated into the specification, making φ' realizable. An illustration of this process is shown in Fig. 2(c), where repair alters the postcondition of a skill from reaching (x_2, y_0) to instead target (x_0, y_0), thereby enabling the robot to satisfy the request.

D. Gait-Free MICP

We aim to implement the suggested skills by reconfiguring the contact sequence and timing to generate new locomotion gaits. This can be achieved by solving a gait-free version of MICP in Eq. (1). The gait-free MICP retains all continuous decision variables and constraints from the gait-fixed MICP but modifies the contact state-dependent constraints, as the contact sequence and timing are no longer fixed. Instead of using $\mathbf{H}_{r,k,j}$, we introduce a different set of binary variables $\mathbf{H}_{r,m,j}$ for defining the contact state for each foot at each time step explicitly. r and j still represent the r^{th} convex region and j^{th} foot, respectively, while $m \in [M]$ denotes the m^{th} time step, evenly discretized with Δt_m over the entire M time steps. We use $\mathcal{O}_{m,m+1}$ to denote the set of continuous time steps that span from the m^{th} to the $m+1^{\mathrm{th}}$ binary time step. As a result, the following constraints are modified:

1) Safe Region Constraint: The safe region constraint is defined in Eq. (21) - (22) similar to the gait-fixed case in Eq. (6) - (7) when a binary variable $\mathbf{H}_{r,m,j}$ is activated. Different from Eq (8), the summation of the binary variables at m^{th} time step for the j^{th} foot is allowed to be zero to

generate a swing phase as shown in Eq. (23).

$$\forall r \in [R], m \in [M], j \in [n_f]$$
$$\mathbf{H}_{r,m,j} \Rightarrow \mathbf{A}_r \mathbf{p}_j[i] \leq \mathbf{b}_r, \forall i \in \mathcal{O}_{m,m+1} \quad (21)$$
$$\mathbf{A}_{\text{eq},r} \mathbf{p}_j[i] = \mathbf{b}_{\text{eq},r}, \forall i \in \mathcal{O}_{m,m+1} \quad (22)$$
$$\sum_{r=0}^{R-1} \mathbf{H}_{r,m,j} \leq 1 \quad (23)$$
$$\mathbf{H}_{r,m,j} \in \{0, 1\} \quad (24)$$

2) Frictional and Contact Constraints: Different from the gait-fixed case in Eq. (10) - (13), the activation of frictional and contact constraints is purely decided by the binary variables.

$$\forall r \in [R], m \in [M], j \in [n_f]$$
$$\mathbf{H}_{r,m,j} \Rightarrow \mathbf{f}_j[i] \cdot \mathbf{n}_r(\mathbf{p}_j^{xy}[i]) \geq 0, \forall i \in \mathcal{O}_{m,m+1} \quad (25)$$
$$\mathbf{f}_j[i] \in \mathcal{F}_r(\mu, \mathbf{n}_r, \mathbf{p}_j^{xy}[i]), \forall i \in \mathcal{O}_{m,m+1} \quad (26)$$
$$\sum_{r=0}^{R-1} \mathbf{H}_{r,m,j} = 1 \Rightarrow \dot{\mathbf{p}}_j[i] = 0, \forall i \in \mathcal{O}_{m,m+1} \quad (27)$$
$$\sum_{r=0}^{R-1} \mathbf{H}_{r,m,j} = 0 \Rightarrow \mathbf{f}_j[i] = 0, \forall i \in \mathcal{O}_{m,m+1} \quad (28)$$

Compared to gait-fixed MICP, gait-free MICP introduces more binary variables to determine contact states, resulting in longer computation times. As such, it is only triggered after performing symbolic repair.

VI. Online Execution

Due to the inevitable disparity between offline synthesis and real-world online terrain conditions, such as variations in terrain shape and position, additional efforts are required to bridge this gap and prevent potential execution failures. The online execution module takes in terrain states $\sigma_{\text{terrain}} \subseteq \mathcal{I}_{\text{terrain}}$ after abstracting the terrain polygons online, a request state $\sigma_{\text{req}} \subseteq \mathcal{I}_{\text{req}}$, and a strategy $\mathcal{A}$ from the offline synthesis module (Sec. V). This module then executes the strategy automaton $\mathcal{A}$ by solving a MICP problem online again with the actual perceived terrain polygons, potentially different from the ones abstracted offline, to generate a reference trajectory for each transition (Sect. VI-A). This reference trajectory along with the corresponding contact information will be further tracked by a tracking controller in real time (Sec. VII). As shown by the blue lines in Fig. 3(c), if the robot encounters an unseen terrain state, we leverage online symbolic repair to create new skills that handle the unexpected terrain state and failure transition at runtime (Sec. VI-B). After reaching the request state σ_{req}, the robot resets the strategy with a new terrain state, request goal states, and repeats the execution until reaching the final goal state.

A. Strategy Automaton Execution and Online MICP Modifications

The red path in Fig. 3(c) represents the automaton online execution process. Before transitioning to the new symbolic state, an online gait-fixed MICP is solved according to the skill provided by the automaton. The automaton advances only if the MICP successfully finds a solution. Slightly different from the gait-fixed MICP formulation during the offline phase, the online gait-fixed MICP is executed given terrain information from a terrain segmentation module that produces labeled convex polygons. For example, a skill transitioning from flat terrain to high terrain, as shown in Fig. 5(a), uses predefined, homogeneous terrain polygons for feasibility check during offline synthesis. Given that, Fig. 5(b) illustrates how the terrain polygons detected online for the same skill may differ from the offline ones in both position and geometry. We emphasize that perception is intentionally treated as out of scope in this paper. We assume an upstream segmentation pipeline (e.g., elevation-mapping with plane segmentation miki2022elevation,fankhauser2018robust) provides labeled convex polygons; in our hardware experiments we use motion-capture-aided manual labeling of polygons, which is sufficient to validate the planning and control framework but does not represent a fully autonomous perception stack. Obtaining convex collision-free regions in unstructured environments is non-trivial and we make no claim that our framework includes a solution to this perception problem. Robust perception integration is left for future work. In addition, the following modifications are highlighted when attempting to transition between two symbolic states in the automaton.

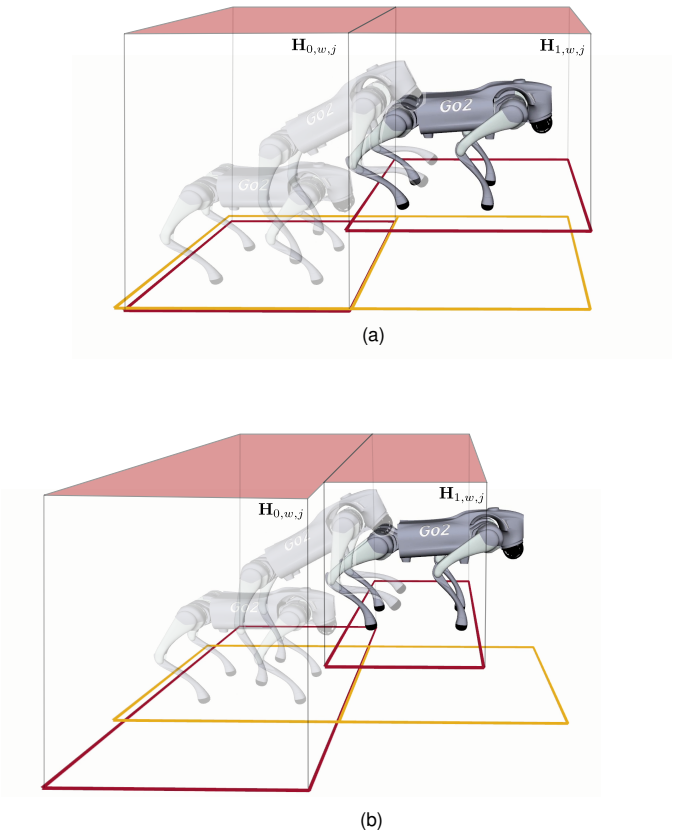


Fig. 5. Demonstrations of (a) offline MICP for a skill transitioning from a flat terrain to a high terrain with predefined polygons; (b) online MICP for the same skill but with online terrain polygons.

1) Robot Pose Re-Targeting: Since the final targeting condition significantly influences the reference trajectory and the feasibility of the MICP problem, we evaluate the final condi-

tion by solving a kinematic feasibility problem (a simplified MICP in Eq. (1)) before solving the online MICP. Compared with the gait-fixed MICP, the dynamics equation (3) is replaced by enforcing the center of mass (CoM) to lie inside a convex support polygon with all feet in stance to ensure static stability. In addition, a hard constraint is added to ensure that the modified robot pose stays within a certain threshold compared with the original desired pose. The cost function is simplified to stay as close as possible to the original desired pose. The higher-order terms of body position, orientation, and end-effectors are excluded from the decision variables. All the other constraints not involving those higher-order terms remain the same. Once the kinematic feasibility problem is solved successfully, the online MICP is solved using the modified re-targeted final condition and reference trajectory.

2) Collision Avoidance and Swing Foot Constraints: Our framework handles obstacles at *two distinct levels*, and it is important to be explicit about which level is responsible in which scenario.

- *Symbolic-level obstacle avoidance.* When an obstacle is large enough that an entire abstracted cell becomes unsteppable, it is classified as the *obstacle* terrain class in the local symbolic abstraction (Sec. III-A). The symbolic specification rules out transitions into such cells, and if an obstacle appears at runtime—as in the rebar-with-obstacle hardware experiment, where the obstacle is detected via motion capture and added to the local abstraction online—runtime symbolic re-synthesis (Sec. VI-B) finds a detour around it. No collision-region variables are added to the online MICP in this case.
- *MICP-level collision-free region selection (this subsection).* For smaller obstacles that the swing foot must clear within a single symbolic transition, we encode collision avoidance directly in the online MICP. Because the resulting trajectory is sent directly to the tracking controller, the quality of end-effector (EE) trajectories at this layer is critical for tracking performance. We introduce binary variables $\mathbf{H}_{s,w,j}$ that constrain the swing-foot trajectory of foot j to lie inside one of S pre-labeled collision-free convex polytopes at each time step. These free-space polytopes come from the same labeled-polygon segmentation pipeline as the steppable terrain, with a separate label set distinguishing free-space polytopes from steppable terrain polygons. In simulation, this mechanism is exercised in the *Unstructured–8* case with collision avoidance enabled (Table IV), where the binary-variable count rises to 256.

We do not perform full-body, mesh-level collision checking against arbitrary geometry; the body footprint is implicitly handled by steppable-region selection keeping each foot inside a labeled terrain polygon. The formulation here trades fidelity for online tractability.

Specifically, $s \in [S]$ represents the s^{th} convex collison-free region and $w \in [W]$ denotes the w^{th} time step, evenly discretized with Δt_w over the entire W time steps. Fig. 5(b) demonstrates two collision-free regions.

$$\forall s \in [S], w \in [W], j \in [n_f]$$
$$\mathbf{H}_{s,w,j} \Rightarrow \mathbf{A}_s \mathbf{p}_j[i] \leq \mathbf{b}_s, \forall i \in \mathcal{C}_{w,w+1} \tag{29}$$
$$\sum_{s=0}^{S-1} \mathbf{H}_{s,w,j} = 1 \tag{30}$$
$$\mathbf{H}_{s,w,j} \in \{0,1\} \tag{31}$$

where $\mathcal{C}_{w,w+1}$ denotes the set of continuous time steps that span from the w^{th} to the $w+1^{\text{th}}$ binary time step. We emphasize that the collision-free region selection in Eq. (29) and the steppable-region selection in Eq. (6) handle different geometric primitives: the steppable-region constraint forces *stance-foot* placement onto a labeled steppable terrain polygon at the first stance time step, while the collision-free region constraint here keeps the *swing-foot* trajectory inside a free-space polytope across all time steps within a swing phase. The two are complementary, not redundant.

To enable larger swing foot clearance from the terrain, we also add constraints to force the swing foot height above a user-defined threshold h_{swing}, given the fixed contact timing. This constraint targets cleanly clearing the terrain during a transition—for example, when jumping onto a higher platform. While such clearance could in principle be enforced through the collision-free region selection alone, doing so would require a fine time-step discretization of the binary variables and would tend to yield trajectories that only marginally clear the terrain. The height threshold h_{swing} instead provides a direct, continuously enforced clearance margin that is straightforward to tune.

Note that the collision-avoidance and swing-foot constraints are also incorporated in the offline MICP as part of the feasibility-checking rules for the relevant terrain transitions. We highlight them here because their impact on online tracking performance is substantially more critical.

B. Runtime Repair

The runtime repair procedure takes as input a terrain state $\sigma_{\text{terrain}} \in \Sigma_{\text{terrain}}$, a request state $\sigma_{\text{req}} \in \Sigma_{\text{req}}$, and a Boolean formula $\varphi_{\text{disallow}}$ that encodes disallowed transitions identified at runtime. We first update the system's hard constraint $\varphi_{\text{s}}^{\text{t}}$ in the specification φ (see Sec. V-B) to $\varphi_{\text{s}}^{\text{t}} \wedge \square\varphi_{\text{disallow}}$, ensuring that the specification reflects these newly discovered restrictions. Next, the high-level manager constructs a partial evaluation of φ over σ_{terrain} and σ_{req}. Symbolic repair is then applied to generate additional robot skills and synthesize a new strategy that accommodates the current terrain and request states, following the approach in Sec. V-C.

VII. Tracking Control

The tracking control module adopts an MPC-Whole-Body-Control (WBC) hierarchy, ensuring precise end-effector (EE) and centroidal momentum tracking. To smoothly and efficiently execute the strategy considering the potential computational delay in solving MICP, a harmonic coordination module between the strategy automaton roll-out and the tracking module is designed (Sec. VII-B).

A. Tracking Control Module

1) Nonlinear Model Predictive Control (NMPC): The NMPC operates independently from the symbolic planning module, tracking reference trajectories generated by the online MICP using a more accurate dynamics model. In this work, the NMPC accounts for the robot's centroidal dynamics and kinematics, similar to approaches in dai2014whole,chignoli2021humanoid,sleiman2021unified, presenting a more accurate model than the simplified angular dynamics in the MICP. An analytical inverse kinematics solution based on the MICP output provides the full joint state reference trajectory for the NMPC. In addition to standard frictional and contact constraints and base tracking costs, an additional cost function for EE reference tracking from the MICP is included to enhance precision. The NMPC optimization problem is formulated as a Sequential Quadratic Program (SQP) and solved using the OCS2 library OCS2, with a time horizon of one second and 100 knot points.

2) Whole-Body Control (WBC): While NMPC operates at the centroidal level to generate dynamically feasible trajectories, WBC ensures precise execution by resolving full-body state control, including joint torques, accelerations, and contact forces. The whole-body controller tracks the NMPC trajectory by solving a weighted quadratic program (QP) kuindersma2014efficiently at 500 Hz. To improve the performance of agile motions such as leaping and jumping, centroidal momentum tracking wensing2013generation is included. This MPC-WBC hierarchy allows the system to handle both centroidal-dynamics-level and full-body-dynamics-level control in a coordinated manner, ensuring robustness in real-world deployment especially for highly dynamic motions such as jumping.

3) State Estimation: The state estimator employs a linear Kalman filter similar to that in bledt2018cheetah, with the addition of OptiTrack motion capture (mocap) measurements fused alongside IMU data and joint encoder readings to achieve accurate body position estimation. For agile motions such as jumping, we observed improved base-height estimation by assigning higher noise values to the mocap feedback during aerial phases, due to its limited 120 Hz update rate.

B. Coordination between Strategy Execution and Tracking Module

Due to the potential delays in solving MICP during the strategy automaton roll-out, a harmonic coordination between the strategy execution and tracking control module is essential. The tracking module's reference trajectory is updated dynamically alongside the strategy execution process, as shown in Fig. 6. The overall process can be summarized as follows:

1) Strategy Execution: The strategy automaton execution thread begins by solving the online gait-fixed MICP, starting from State 0 in the automaton. Once the MICP solution is obtained, it immediately sends the reference trajectories and corresponding contact information to the tracking control module. The automaton then progresses to the next transition, using the final condition in the previous trajectory segment as the new initial condition. In Fig. 6, the colored blocks in the strategy execution timeline indicate the time taken for each MICP solve during the automaton roll-out. PT and TT denote the planning time and trajectory time, respectively.

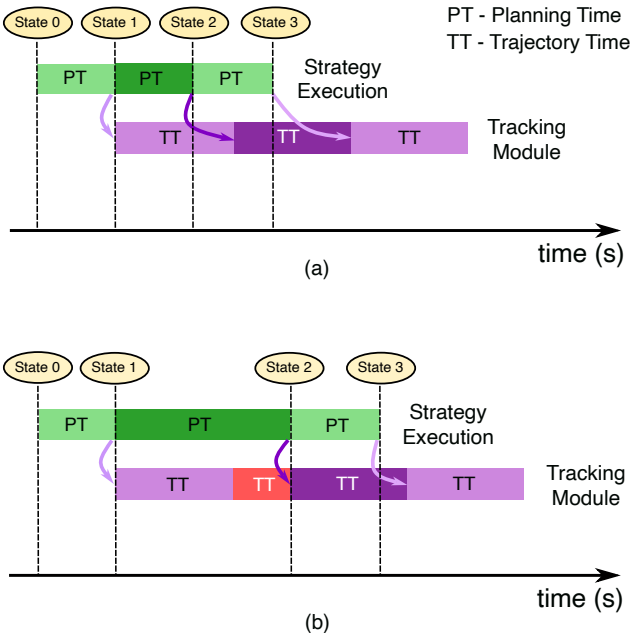


Fig. 6. Coordination between strategy execution and tracking control module. (a) The MICP solving time (denoted by PT) for transitioning from State 1 to State 2 is shorter than the time horizon of the previous reference trajectory (denoted by TT), allowing the seamless appending for the new trajectory. (b) The MICP solving time exceeds the time horizon of the previous reference trajectory, requiring the robot to come to a stop (red) and wait for the new trajectory whenever available.

2) Tracking Control Module: The MPC-WBC tracking control module receives time-indexed reference trajectories, which can be updated in real-time. After completing the tracking of the current reference trajectory, the robot returns to a stance phase, ready to accept new reference trajectories. As depicted in Fig. 6, the colored blocks in the tracking control module represent the time horizon of each reference trajectory sent by the strategy execution thread. Once the initial reference trajectory is generated, the MPC-WBC thread begins tracking it using more accurate dynamics models, as described in Sec.VII.

3) Delay Handling: For more complex scenarios involving a larger number of terrain polygons, the MICP solving time may increase and exceed the time horizon of the generated reference trajectory. Fig. 6(a) and (b) illustrate two cases of MICP solving times: Case (a): The solving time for transitioning from automaton State 1 to State 2 is shorter than the time horizon of the previous reference trajectory. In this case, the new reference trajectory is seamlessly appended to the existing one, and the robot continues tracking the previous trajectory. Case (b): The solving time exceeds the time horizon of the previous reference trajectory, and the robot has already entered a stance phase (red block) when the MICP solution is obtained. In this case, the robot waits until the newly generated reference trajectory is sent based on the current system time (red block in Fig. 6 (b)).

Fig. 7. Hardware platforms used for experiments: Unitree Go2 and SkyMul Chotu (equipped with a rebar gun and "cross"-shaped feet).

TABLE II
ROBOT PARAMETERS

Parameter	Unitree Go2	SkyMul Chotu
m (kg)	15	20
$\boldsymbol{\tau}_{\max}$ (N · m)	(23.5, 23.5, 45.4)	(23.5, 23.5, 33.5)
$\mathbf{q}_j^{\text{ref}}$ (rad)	(0.0, 0.72, -1.44)	(0.0, 0.72, -1.44)
${}^{\mathcal{B}}\mathbf{p}_j^{\text{ref}}$ (m)	(0.1805, 0.1308, -0.29)	(0.2118, 0.210, -0.30)
$\mathbf{p}_j^{\max}$ (m)	(0.15, 0.1, 0.15)	(0.15, 0.1, 0.15)
$\mathbf{I}$ (kg · m^2)	diag(0.152, 0.369, 0.388)	diag(0.396, 0.915, 1.107)

VIII. SIMULATION

We present examples of maneuvering across diverse environments in a Gazebo simulation to demonstrate the framework's efficacy, scalability, and generalizability as in zhou2025physically. In addition, benchmarking experiments are conducted to further compare our proposed framework with pure MIP approaches.

A. Implementation Details

All MICP problems—including the offline gait-free MICP, the online gait-fixed MICP, the kinematic-feasibility retargeting MICP, and the pure-MIP benchmark—are solved using Gurobi 9 gurobi. The NMPC tracker is implemented with the OCS2 library OCS2. All offline-synthesis, simulation, and online-planning experiments are run on a desktop workstation equipped with an Intel Core i9-13900K (24 cores) and 64 GB of RAM. Reported solve times reflect this desktop-class CPU and should be interpreted as such; onboard, low-power deployment is left as future work.

B. Robot Setup

As shown in Fig. 7, we verify our algorithms on two separate robot platforms, Unitree Go2 and SkyMul Chotu (a modified Unitree Go1 robot dedicated for rebar tying tasks). The SkyMul Chotu is equipped with a rebar gun mounted on its head and customized "cross"-shaped feet designed for reliable standing on rebars. Note that in simulation, this foot shape is simplified to a regular round foot to avoid the complexity of simulating contact with irregular surfaces. In our planner, we model both cases as point contact. The parameters for each robot are defined in Table II and used in our implementation, including the robot mass m, torque limit $\boldsymbol{\tau}_{\max}$ and the reference joint position $\mathbf{q}_j^{\text{ref}}$ for a single leg with three joints, the reference foot position relative to the base frame ${}^{\mathcal{B}}\mathbf{p}_j^{\text{ref}}$ and the maximum deviation of the foot position $\mathbf{p}_j^{\max}$ for the front left leg ($j = 0$), and the moment of inertia of the approximated single rigid body $\mathbf{I}$.

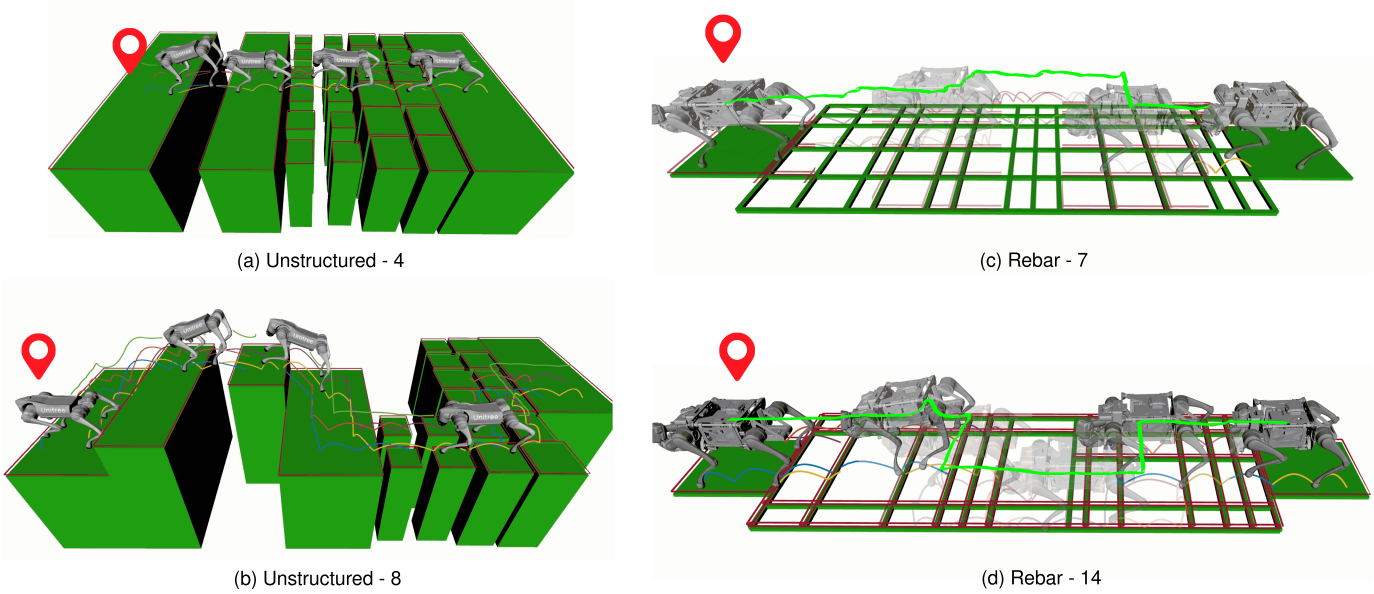
C. Simulation Environment Setup

To demonstrate the generalizability of our proposed framework, we evaluate it across two scenarios involving different terrain types, safe stepping regions, and robotic platforms—Unitree Go2 and SkyMul Chotu. Obstacles are modeled as a distinct terrain class, and obstacle avoidance is encoded as hard constraints within $\varphi_{\text{s}}^{\text{t}}$. The requested waypoints are assumed to be free of obstacles. We test both 3×3 and 5×5 grid abstractions. The 5×5 grid enables a longer planning horizon but introduces higher decision complexity. For discretization, we use a cell size of 0.8 m in unstructured terrains and 0.6 m for the rebar case. These values can be adjusted based on the requirements of specific deployment environments.

1) Unstructured Terrain: We abstract 8 terrain types—*flat terrain*, *high terrain*, *low terrain*, *dense stone*, *sparse stone*, *gap*, *high gap*, and *low gap*—based on classification criteria involving polygon heights, counts, and areas relative to the abstracted grid cells. A terrain is categorized as a *gap* when the ratio of its overlapping area with a grid cell falls below a specified threshold. We define two terrain configurations: one consisting of four selected terrain types, and another comprising all eight. These are illustrated in Figs. 8(a) and 8(b), respectively.

2) Rebar Terrain: Inspired by efforts to automate labor-intensive rebar tying tasks on construction sites asselmeier2024hierarchical, we investigate a second case study in which a quadrupedal robot navigates a rebar mat. Each rebar is modeled as a rectangular polygon with a 3 cm width. Unlike the stepping stone scenario, this setup presents a denser distribution of potential footholds due to the higher rebar density. Notably, the robot's traversability depends on the directional sparsity of the rebars—whether aligned with or orthogonal to the robot's facing direction—within an acceptable tolerance.

We abstract and classify rebar types based on horizontal sparsity (perpendicular to the robot's facing direction) and vertical sparsity (parallel to it). Within each abstracted cell, the number and spacing of rebars are evaluated to classify the sparsity in each direction as *dense* (0.05–0.15 m), *sparse* (0.15–0.35 m), *extreme sparse* (above 0.35 m), *single*, or *none*. To reduce complexity, combinations deemed too challenging—such as *none* or *single* in both directions—are treated as *obstacle*. Based on this abstraction, we define two example configurations comprising 7 and 14 rebar terrain types. The 7-type configuration in Fig. 8(c) samples rebar sparsity randomly between 0.15 and 0.35 m, omitting *extreme sparse* regions. In contrast, the 14-type configuration in Fig. 8(d) includes sparsity ranging from 0.15 to 0.6 m, encompassing more complex and difficult combinations including *extreme sparse* types.



(a) Unstructured - 4 (c) Rebar - 7

(b) Unstructured - 8 (d) Rebar - 14

Fig. 8. Unstructured and rebar terrain scenarios (presented in zhou2025physically as well).

TABLE III
OFFLINE SYNTHESIS AND REPAIR TIME

Scenario	Grid Size	Terrain and Request State Pairs (Success/Total)	Number of Skills (Original/New/Total)	Symbolic Repair Time (s)	Feasibility Check Time (s) (Gait-Fixed/Gait-Free)
Unstructured - 4	3×3	169/169	18/13/64	8.07	18.97/449.94
	5×5	129/158	19/9/64	77.16	20.26/241.42
Unstructured - 8	3×3	250/264	19/20/256	14.90	22.92/382.63
	5×5	179/246	20/16/256	93.21	18.59/196.88
Rebar - 7	3×3	196/196	18/11/196	7.44	15.63/417.55
	5×5	196/196	18/10/196	21.63	14.83/374.89
Rebar - 14	3×3	237/237	20/29/784	10.34	21.92/701.38
	5×5	196/196	20/18/784	24.31	23.06/453.94

TABLE IV
ONLINE GAIT-FIXED MICP SOLVING TIME FOR SIMULATION

Scenario	Collision Avoidance	Binary Variables	Continuous Variables	Number of Polygons	Solve Time (s)
Unstructured - 4	No	52	6583.5	6.5	0.37
Unstructured - 8	Yes	256	5166	2	2.65
	No	69	7938	7.5	0.49
Rebar - 7	No	70	7938	7.875	1.11
Rebar - 14	No	58	8142.75	6.25	1.09

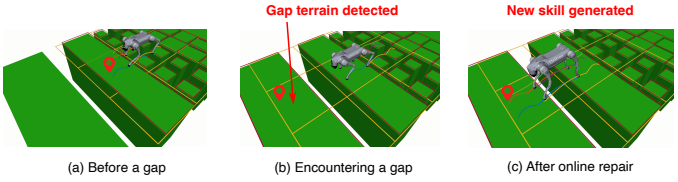


Fig. 9. Runtime repair scenario when encountering a gap.

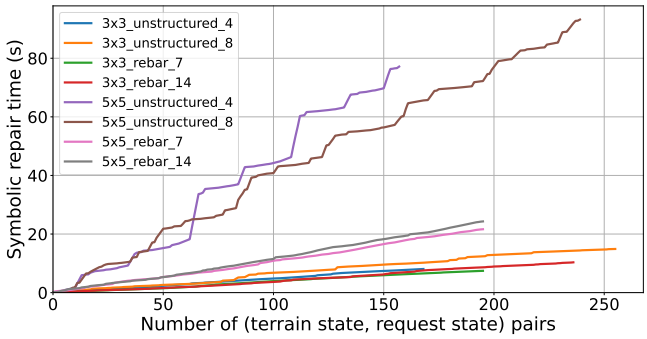


Fig. 10. Symbolic repair time in the size of terrain and request state pairs.

D. Offline Synthesis Results

We evaluate the offline synthesis module for both *Unstructured* and *Rebar Terrain* scenarios by initializing the skill set with two trotting gaits (2 s and 3 s). The inclusion of the longer-horizon trotting gait provides additional safer walking options from a practical standpoint. Prior to synthesis, we construct the set of possible terrain states $\Sigma_{\text{terrain}}^{\text{poss}}$—assumed to be provided by the user—by systematically sweeping a local grid over the entire terrain map, based on the best available prior knowledge. The set of request states $\Sigma_{\text{req}}^{\text{poss}}$ is defined as the top row of the local grid in the robot's facing direction,

with the two corner cells also included to enable movement in diagonal directions.

Table III reports (i) the number of terrain and request states pairs $(\sigma_{\text{terrain}}, \sigma_{\text{req}}) \in \Sigma^{\text{poss}}_{\text{terrain}} \times \Sigma^{\text{poss}}_{\text{req}}$, including both successful and total ones, (ii) the number of skills, including the original, the newly discovered, and the total possible ones, and (iii) offline repair and feasibility checking times, including both from gait-fixed and gait-free MICP. We first observe that the repair process successfully resolves 93.4% of the terrain and request state pairs. The remaining cases are deemed unrepairable due to either unreachable request states caused by obstacles or inherent physical infeasibility. As highlighted in red, the number of newly discovered skills is reduced by 71.6–97.6% compared to exhaustively checking all possible skills—each of which requires solving a computationally expensive gait-free MICP. On average, generating a new locomotion gait via gait-free MICP takes 27.23 s, with a maximum of 145.66 s. This demonstrates the effectiveness of offline repair in substantially reducing the number of costly gait-free MICP solves.

To investigate the scalability of offline repair, we perform repair across various sizes of the terrain and request states. As shown in Fig. 10, the repair runtime grows linearly in the size of the terrain and request states for both 3×3 and 5×5 grids. While fewer terrain and request states handled offline reduce checked time, they may lead to more frequent unforeseen states during online execution. Moreover, we note that the slope of 5×5 cases in Fig. 10 are steeper than those of 3×3 as a result of the increased state space. On average, repair takes 478.1% more time for 5×5 grids than 3×3 for one terrain and request states pair. In practice, the perception range and the robot size limit the grid size, so we do not test beyond 5×5 grids, though the user can adjust the resolution.

E. Online Execution Results

Fig. 8 presents snapshots of Go2 and Chotu navigating various terrains during online execution. The robot is tasked with reaching a global waypoint using a naive global shortest-path planner. At each step, the local waypoint is selected as the closest non-obstacle cell in the local grid map pointing toward the global target.

In Fig. 8(a), the robot traverses a sequence of terrain types including *flat terrain*, *dense stepping stone*, *sparse stepping stone*, and *gap*. Fig. 8(b) showcases a more complex maneuver involving all eight terrain types, including elevation changes, where the robot adaptively selects suitable locomotion gaits. Similarly, Figs. 8(c) and (d) illustrate Chotu traversing rebar terrains with 7 and 14 defined rebar types, respectively. Notably, the robot utilizes newly discovered leaping gaits with short aerial phases to execute challenging transitions—such as moving from lower to higher elevations or crossing over gaps and *extreme sparse* rebar segments, as demonstrated in Figs. 8(b) and(d).

Table IV summarizes the average number of decision variables, terrain polygons, and solve time for each scenario, computed across all step transitions in a full locomotion run with the online gait-fixed MICP. On average, the online gait-fixed MICP takes 430 ms to solve the unstructured terrain cases and 1.1 s for rebar cases when the collision avoidance is disabled. The rebar scenarios exhibit 155.8% longer solve times due to their overlapping, skewed rebar arrangement, and a larger number of terrain polygons. Additionally, enabling collision avoidance (necessary for *Unstructured - 8* case to handle the elevation change) increases the number of binary variables by 271.0%, increasing the solve time by 440.8% (highlighted in red).

F. Runtime Repair Case Study

As a case study to demonstrate the framework's capability to handle unforeseen terrains and online failures, we highlight the following run-time scenarios that trigger runtime repair or re-synthesis.

1) Unforeseen Terrain States: With limited prior knowledge of the global terrain map, the terrain states provided during offline synthesis may be insufficient when encountering new terrain configurations. Fig. 9 illustrates this in an *Unstructured - 4 types* scenario with a 3×3 grid size, where terrain states involving *gap* terrain are deliberately excluded from the offline phase. When the robot reaches a new terrain state that contains a *gap*, runtime repair is triggered for the current terrain and request states. A new skill with a leaping gait is generated to successfully cross the gap. The runtime repair takes 12.36 s in total, of which the symbolic-repair stage accounts for 0.18 s and the gait-free MICP for the new leaping gait accounts for 12.18 s—confirming that the dominant cost lies in the dynamic-feasibility check, not the symbolic search.

2) Solving Failure: Another common runtime failure arises from MICP solving failures. Due to discrepancies between the predefined polygons used during offline synthesis and the actual online terrains—such as variations in shape and size—a skill deemed feasible offline may become infeasible when executed online, even if classified under the same symbolic transition. The detours shown in Fig. 8(c) and (d) illustrate the re-synthesis process triggered by solving failures in the 7-type and 14-type rebar cases. Solving failures occur more frequently in the rebar scenarios than in the unstructured ones, due to the higher uncertainty in the shapes and sizes of online rebar polygons. Re-synthesis takes 0.11 s and 0.07 s respectively for the two cases. Neither case triggers full runtime repair, because the existing skill set suffices to navigate to the goal under the failed transition.

G. Benchmark: Pure MIP Planner

To evaluate how the symbolic planning benefits the overall planning framework, we benchmark our method against pure MIP approaches, where we implement a planner to evaluate how computation time scales with the planning horizon and terrain complexity. Specifically, we examine five planning horizons ranging from 1 s to 5 s. In addition, we vary the terrain types by considering sparse stepping stones (0.15 m spacing), dense stepping stones (0.05 m spacing), sparse rebar (0.3 m spacing), and dense rebar (0.15 m spacing), as illustrated in Fig. 11 with specific colors.

To make the comparison precise: the pure-MIP baseline uses the *same* convex relaxations of the dynamics (Eq. (3)),

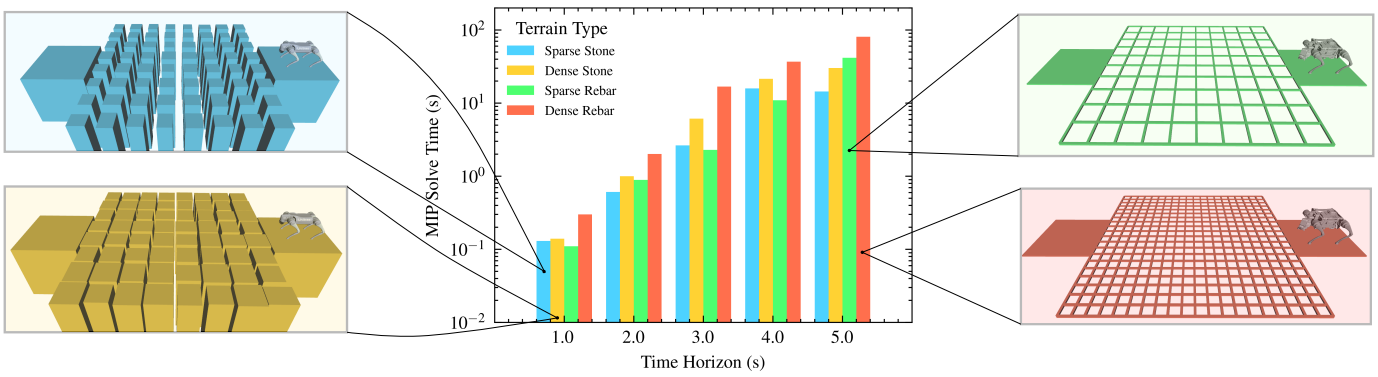


Fig. 11. MIP solve time benchmark. Four scenarios including sparse stepping stone (blue), dense stepping stone (yellow), sparse rebar (green), and dense rebar (red) are evaluated with time horizons ranging from 1 - 5 seconds.

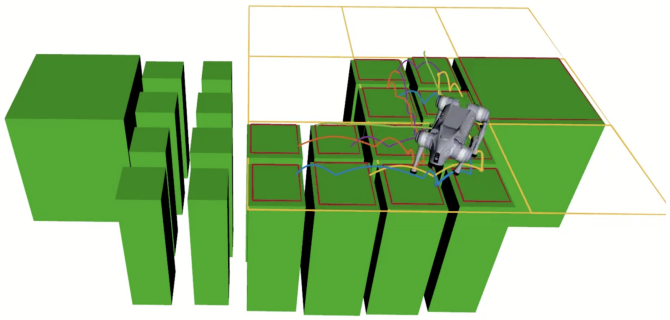


Fig. 12. Demonstration of the robot executing multiple turning behaviors to continuously progress toward the global waypoint on terrain arranged in a zig-zag pattern.

the *same* linearized friction-cone and fixed-Jacobian kinematic constraints, and the *same* cost weights as our online gait-fixed MICP. The only differences are (i) the planner solves a single long-horizon gait-fixed MICP without symbolic-level decomposition, and (ii) the local planning region is expanded to encompass all terrain polygons within the horizon-induced reachable distance, rather than being limited to the cells adjacent to a symbolic transition. The same Gurobi solver and settings are used in both cases. We do not benchmark against a non-convex MIP without relaxations because such a formulation would not provide the global feasibility certificate that makes our gait-fixed MICP comparable to the pure-MIP baseline.

The benchmarked planner is configured to solve only gait-fixed MICP; we omit gait-free MICP due to its significantly longer solve time in longer horizon setup that are more than 2 s. For a given planning horizon, the local planning region is linearly expanded to increase the traversable distance. The locomotion gait is repeated cyclically on a 1 s trotting pattern. During execution, the planner—similar to our proposed framework—receives a global waypoint and incrementally computes local waypoints, selecting the farthest reachable position within the local planning region.

An alternative baseline mentioned in the literature is to repeatedly solve a short-horizon receding-horizon MIP at every NMPC tick. We chose not to include this in our benchmark because, with a fixed gait and convex relaxations identical to ours, the receding-horizon variant amounts to running our online gait-fixed MICP at every NMPC tick—a setting whose per-iteration solve time is captured by the 1 s data point in Fig. 11 (the leftmost bar of each color). Critically, the receding-horizon planner has no symbolic horizon and therefore cannot avoid local minima introduced by long-horizon obstacle layouts (e.g., the gap and *extreme sparse* rebar scenarios in Sec. IX); this is precisely the scenario where our symbolic-level guidance provides qualitative advantages, beyond the per-iteration solve-time argument shown in the figure.

As shown in Fig. 11, the histogram illustrates the average time to solve the MIP in different scenarios and time horizons. The results show that, within the same scenario, the MICP solve time increases almost exponentially with the planning horizon. Furthermore, dense rebar and stepping stone scenarios consistently require longer solve times than their sparse counterparts, due to the larger number of terrain segments involved in the optimization. In our proposed method, we primarily use 2 to 3 s gait horizons for each symbolic transition, striking a balance between computational efficiency and sufficient planning foresight. This choice avoids overly short myopic horizons, while still allowing for adaptive gait behaviors. For the same data, std-dev whiskers across the per-trial samples (omitted from Fig. 11 for visual clarity) are reported in TODO Table TODO so that the reader can assess solve-time variability around each reported mean.

H. Preliminary Study: Yaw Command Extension

This subsection demonstrates that the proposed framework can be extended to non-zero yaw waypoints in principle, but we report the result as a *preliminary feasibility study* only. We do not claim a full demonstration of generalizability. Specifically: (i) the symbolic abstraction in Sec. III-A grows combinatorially when yaw is added as a discretized input, and we have not characterized how the manager and symbolic-repair scaling numbers reported in Sec. VIII change with yaw discretization; (ii) the angular-momentum coupling neglected by the convex-dynamics simplification in Eq. (3) becomes

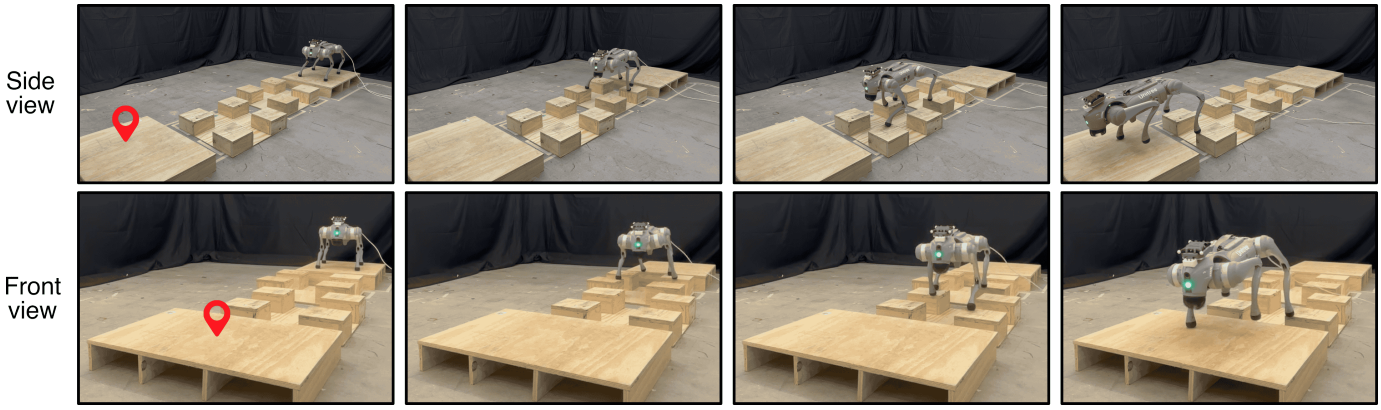


Fig. 13. Hardware experiment demonstration for the first unstructured terrain scenario with sparse stepping stones and flat terrains.

more relevant once yaw is treated as a planned variable, especially for fast turns; (iii) we report simulation results only—no hardware deployment of the yaw extension.

In the previous examples, we assume that the yaw angle of the robot base is fixed to simplify the problem. However, the robot's physical capability to traverse terrains largely depends on the base orientation. For instance, performing a forward jump onto a high terrain is different from a sideway jump, in terms of both leg kinematics and dynamics. Therefore, to demonstrate the framework's modularity, we provide a preliminary extension to the existing abstraction of the robot state to allow yaw variation.

Specifically, the robot inputs are now defined as $\mathcal{I}_{\text{robot}} := \{\pi_x \mid x \in \mathcal{X}\} \cup \{\pi_y \mid y \in \mathcal{Y}\} \cup \{\pi_{yaw} \mid yaw \in \mathcal{W}\}$, where we define a new set of yaw angles $\mathcal{W} := \{-180°, -90°, 0°, 90°, 180°\}$. Note that, a finer discretization of the yaw angles can be defined as needed. Similarly, the request inputs are defined as $\mathcal{I}_{\text{req}} := \{\pi_x^{\text{req}} \mid x \in \mathcal{X}\} \cup \{\pi_y^{\text{req}} \mid y \in \mathcal{Y}\} \cup \{\pi_{yaw}^{\text{req}} \mid yaw \in \mathcal{W}\}$. The robot and the request input can be further divided into translational $\mathcal{I}_{\text{robot}}^{\text{trans}}, \mathcal{I}_{\text{req}}^{\text{trans}}$ and rotational parts $\mathcal{I}_{\text{robot}}^{\text{orien}}, \mathcal{I}_{\text{req}}^{\text{orien}}$, where the translational ones contain the x and y states and the orientational ones contain the yaw state.

The skill is further divided into translational and rotational skills. A translational skill $o_{\text{trans}} \in \mathcal{O}_{\text{trans}}$ consists of sets of preconditions $\Sigma_{o_{\text{trans}}}^{\text{pre}} \subseteq \Sigma_{\text{robot}} \times \Sigma_{\text{terrain}}$ that include the full robot state, and translational-only postconditions $\Sigma_{o_{\text{trans}}}^{\text{post}} \subseteq \Sigma_{\text{robot}}^{\text{trans}}$. A rotational skill in place $o_{\text{orien}} \in \mathcal{O}_{\text{orien}}$ consists of sets of rotational-only preconditions $\Sigma_{o_{\text{orien}}}^{\text{pre}} \subseteq \Sigma_{\text{robot}}^{\text{orien}}$ and postconditions $\Sigma_{o_{\text{orien}}}^{\text{post}} \subseteq \Sigma_{\text{robot}}^{\text{orien}}$. The terrain state can also be incorporated when generating rotational skills, particularly for terrains that pose challenges for turning. However, we omit this in our current implementation for simplicity. The specification encoding then follows the same procedure as described in Sec. V-B, based on each skill's precondition and postcondition. The hard constraints remain unchanged, except that translational and rotational ones are defined separately to ensure the corresponding states remain unchanged when no skill is selected.

To demonstrate the resulting maneuver, we construct a scenario with multiple dense and sparse stepping stones arranged in an overall zig-zag pattern, requiring several turning behaviors. Although sideways movement skills are feasible, we manually exclude them before synthesis to highlight yaw adjustments controlled by the rotational skills. As shown in Fig. 12, the robot executes multiple turns to continuously progress towards the global waypoint. More complex coupled translational and rotational behaviors can also be defined by modifying the preconditions and postconditions associated with each skill.

IX. Hardware Demonstrations

We demonstrate that our entire framework can be successfully deployed on hardware, highlighting two key aspects. First, we showcase the framework's capability to generate adaptive locomotion gaits across diverse terrain types, particularly focusing on agile leaping motions. Second, we illustrate the framework's assured safety through obstacle avoidance and selection of dynamically feasible gaits, benchmarked against a heuristic-based planner.

A. Hardware Experiment Setup

For hardware experiments, we set up similar real-world scenarios for both unstructured and rebar terrains. For the unstructured terrain scenario, we construct wooden blocks to form stepping stones, gaps, and flat regions, with each stepping stone elevated approximately 0.12 m above the ground. For the rebar scenario, we assemble a realistic rebar mat using fourteen $1/2$-inch $\times$ 4-foot rebars and six $1/2$-inch $\times$ 10-foot rebars. In the synthesis setup, obstacles are considered a distinct terrain type, consistent with the simulation. Both scenarios assume the maximum potential terrain types—8 for unstructured and 14 for rebar—though the real setups include fewer types. As demonstrated in the simulation, this does not increase complexity, as it only scales linearly with terrain-request pairs due to our high-level manager. For safety, we set abstraction cell sizes to 0.8 m for the unstructured terrain and 0.45 m for the rebar terrain.

B. Unstructured Terrain

As shown in Fig. 13, we first create a scenario with sparse stepping stones and flat terrain regions. The stepping stones

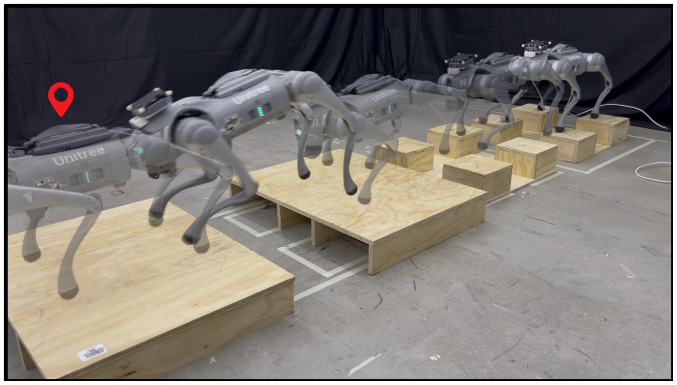


Fig. 14. Hardware experiment demonstration for the second unstructured terrain scenario with sparse stepping stones, flat, and gap terrains.

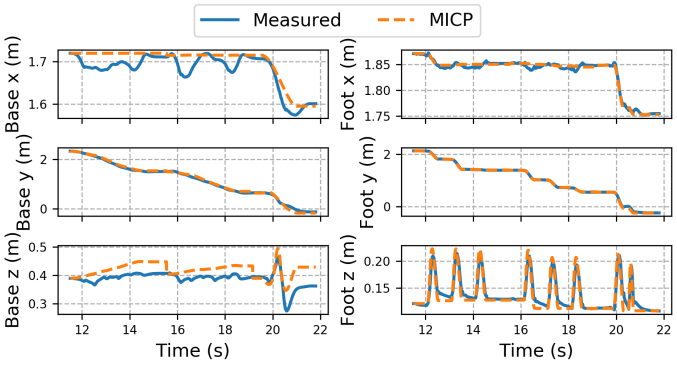


Fig. 15. Tracking performance of the base and front-left foot positions: comparison between measured states and MICP-generated reference trajectories.

are spaced between $0.18 - 0.23$ m apart. Both side and front views are provided to highlight the misalignment of stepping stones along the robot walking direction. The MICP planner successfully generates a reference trajectory involving non-trivial sideways movement, enabling the robot to safely land on the stepping stones. All transitions utilize a 3 s trotting gait.

In the second scenario, shown in Fig. 14, we further include a gap terrain segment. The gap, approximately 30 cm wide, necessitates the use of a leaping gait. Since the gap terrain was incorporated during offline synthesis, our framework can directly select an optimized 1.5 s leaping gait generated by gait-free MICP upon encountering this gap terrain. The task-space tracking performance between the measured state and the reference trajectory generated by the online MICP is shown in Fig. 15. This includes the floating base position and the end-effector (EE) position for the front-left foot. The EE tracking maintains errors within 0.01 m in the x and y axes, ensuring precise foot placement. Meanwhile, the NMPC effectively adjusts the base position to compensate for model simplifications at the MICP level, enabling successful execution of both trotting and leaping motions.

C. Rebar Terrain

For the rebar terrain scenarios shown in Fig. 16, we test two cases: one without any obstacles and one with an obstacle introduced during execution. The rebar spacing varies between 0.1 m and 0.3 m. Offline synthesis is conducted without prior knowledge of obstacles. In the first case (top row), the robot Chotu successfully performs the entire maneuver without encountering any runtime failures, consistently moving forward until reaching the goal waypoint. In the second case (bottom row), an obstacle is added to the environment but only detected during the online execution phase. In our hardware setup, the obstacle's location is not estimated by an autonomous perception stack but rather provided through motion-capture-aided manual annotation: when the obstacle enters the robot's local field of view, its position is added to the online terrain set and the affected cell is classified as the *obstacle* terrain class at the symbolic level—i.e., obstacle avoidance in this scenario is handled entirely at the symbolic level (Sec. III-A), not through the MICP-level collision-region variables $\mathbf{H}_{s,w,j}$ in Eq. (29). This setup is explicit about what is and is not validated—the planning and runtime-repair pipeline is exercised end-to-end, while the terrain-segmentation problem itself is not. Upon encountering a new terrain/request pair caused by the obstacle, the planner promptly triggers a runtime resynthesis. Leveraging previously generated skills, it computes a new strategy within 0.05 s to detour around the obstacle and continue toward the goal. All transitions in both scenarios use a 2.25 s static walking gait, where only one foot is lifted at a time to ensure safety. We note that the gait timings adopted on hardware—3 s trotting, 1.5 s leaping, 2.25 s static walking—are conservative compared to the most agile motions reported in the legged-locomotion literature; however, they exercise the framework's gait-switching, online MICP, and runtime-repair mechanisms, which is the validation target of this article. Aggressive, high-speed gaits would require revisiting the convex-dynamics simplification (Eq. (3)) discussed in Sec. VI.

D. Benchmark: Heuristics-Based Planner

We create a third rebar scenario, shown in Fig. 17, by removing two rebars to further increase the maximum spacing to 0.48 m. This highlights the safety advantage of our planner compared to a heuristics-based footstep planner. The heuristic baseline follows a similar strategy to fankhauser2018robust, kim2020vision, adapted for the rebar scenario by selecting the nearest rebar polygon and the closest feasible foothold to a nominal target. The resulting footstep plan and linearly interpolated base trajectory are sent to the same tracking control module. For fairness, all parameters are tuned to achieve successful traversal at a speed of 0.1 m/s (as shown in the supplemental video).

We chose this nearest-feasible-foothold heuristic because it isolates the value of feasibility-aware, longer-horizon planning over a single-step heuristic, and because such heuristics remain widely used in deployed perceptive-locomotion stacks. More sophisticated optimization-based baselines such as TOWR-style phase-based parameterization winkler2018gait or learning-based perceptive controllers hoeller2023anymal target a different problem (continuous-time gait optimization or end-to-end policy learning), and a fair comparison would require equipping them with a comparable symbolic decomposition; we discuss this as future work in Sec. X.

Without re-performing the offline synthesis, our framework directly uses the results from Sec. IX-C. It classifies the

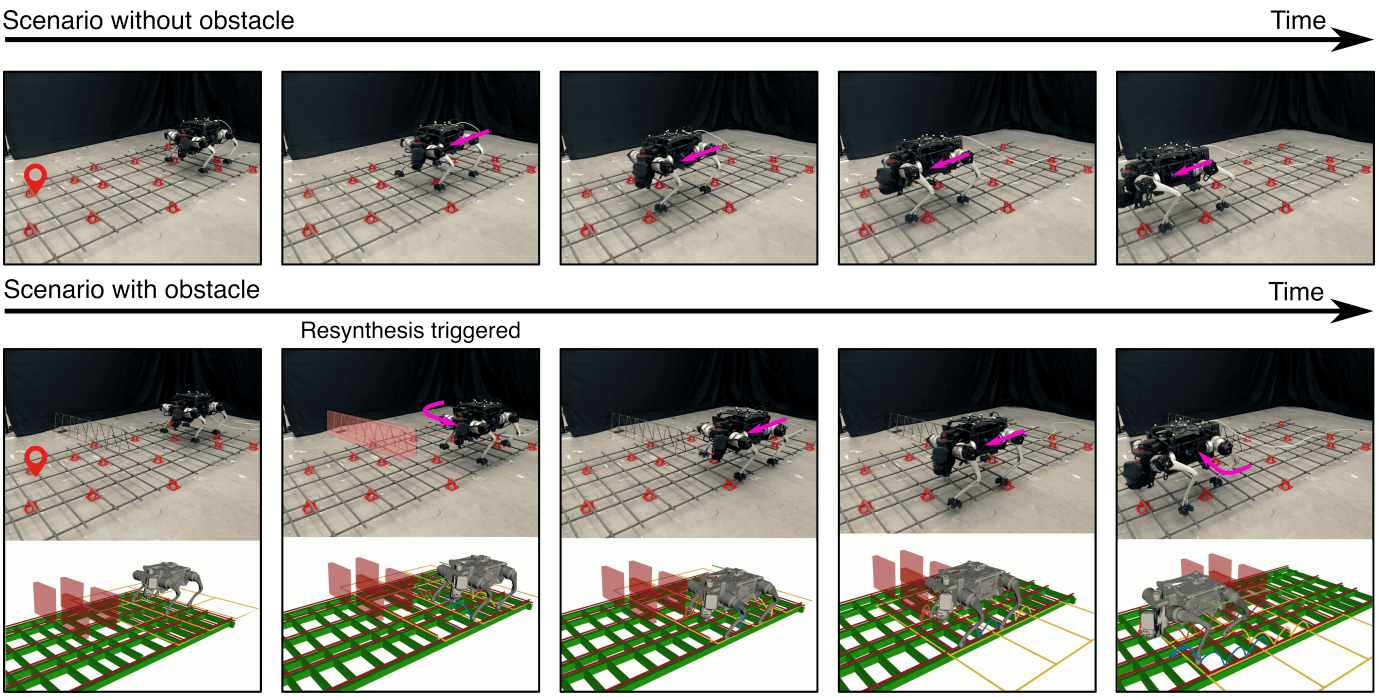


Fig. 16. Execution results in rebar terrain without (top row) and with obstacle (bottom row). In the obstacle case, runtime resynthesis is triggered to generate a detour strategy and ensure successful traversal.

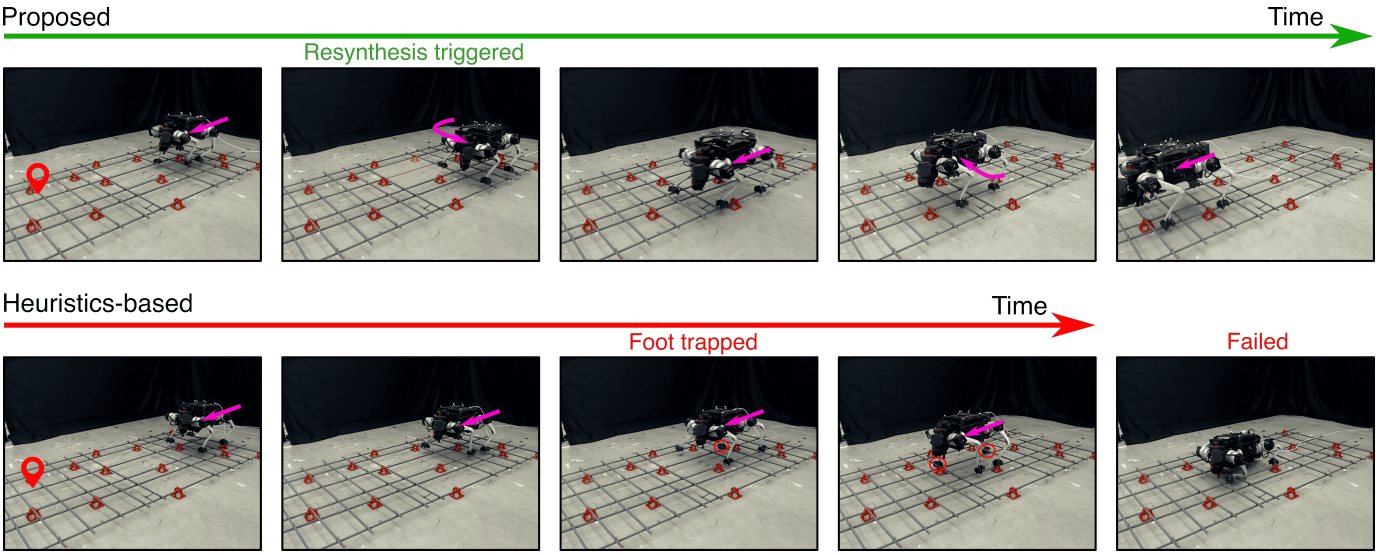


Fig. 17. Comparison between the proposed method and a heuristics-based planner in an extreme sparse rebar scenario. The proposed method performs resynthesis and succeeds, while the heuristics-based planner fails due to foot trapping.

extreme sparse region as an obstacle and performs runtime resynthesis, generating a detour to safely reach the goal. In contrast, we task the heuristics-based planner with the same goal at 0.2 m/s to match our planner's effective gait speed (i.e., 0.45 m per 2.25 s transition). Without reasoning about the compatibility between terrain geometry and gait feasibility, the heuristic planner suffers from poor EE tracking after a few steps, leading to foot slippage and eventual failure as the robot is entangled by the sparse rebar layout.

E. Computational Time

We report the hardware computational time in Table V. Except for the second scenario in the unstructured terrain, we observe a notable increase in solve time compared to simulation. This is mainly due to two factors: (1) In the first unstructured terrain scenario, the two sets of stepping stones are laterally misaligned from the robot's viewpoint, requiring non-trivial deviation from the nominal foot location and sideways movement; (2) In the rebar scenarios, the real-world rebar polygons are not perfectly aligned with the x or y-axis as in simulation, introducing additional numerical complexity for

TABLE V
ONLINE GAIT-FIXED MICP SOLVING TIME FOR HARDWARE

Scenario	Polygons	Min (s)	Mean (s)	Std (s)	Max (s)
Unstructured - Scene 1	6	0.81	4.04	TODO	7.86
Unstructured - Scene 2	5	0.16	0.53	TODO	0.96
Rebar - Scene 1	8	3.88	6.54	TODO	9.26
Rebar - Scene 2	7.5	3.60	9.13	TODO	19.23
Rebar - Scene 3	6.33	2.45	7.37	TODO	14.7

branch-and-bound solvers. Further acceleration of MIP solving through tighter convex relaxations marcucci2024shortest, lin2024accelerate remains an important direction for future work.

X. DISCUSSION

A. Generalization to More Types of Terrain

The current framework relies on manual terrain abstraction and classification, which requires expert knowledge to define terrain types and assign them to discrete symbolic categories. This process typically involves analyzing physical features such as terrain shape, height variation, and support area to determine whether a terrain segment should be labeled as flat, sparse, dense, high, gap, etc. While effective, this expert-driven approach may limit scalability and generalization to novel environments or terrain conditions. Future extensions could incorporate data-driven terrain classification and generation methods, such as supervised learning or clustering based on 3D point cloud statistics to automate the terrain abstraction process and reduce reliance on human heuristics.

B. Optimality

The synthesis process focuses on guaranteeing feasibility and correctness of transitions but does not account for the relative cost or quality of different feasible gaits. When multiple locomotion gaits (e.g., trotting, bounding, leaping) are viable for a given symbolic transition, the current method does not evaluate their optimality in terms of energy efficiency, robustness, execution time, or other performance metrics. Instead, it selects the first gait that satisfies the physical feasibility conditions via MICP. Integrating cost-aware synthesis would enable the framework to prioritize higher-quality behaviors and make more informed decisions under trade-offs.

C. Perception Module

The proposed framework is modular and can be directly integrated with a wide range of perception pipelines that provide terrain geometry in the form of segmented polygonal regions miki2022elevation. Existing perception modules that perform terrain segmentation using LiDAR, RGB-D sensors, or stereo vision can be adapted to supply the required input to the MICP planner and symbolic abstraction layers. In this work we deliberately treated the perception layer as out of scope and used motion-capture-aided manual labeling on hardware (Sec. IX); a fully autonomous deployment will need to address how perception uncertainty—segmentation noise, polygon misalignment, and per-frame perception delay—propagates through the MICP feasibility certificates and the symbolic-level realizability guarantees. In the current framework, runtime symbolic repair already provides a partial fallback: when perceived terrain causes the symbolic specification to become unrealizable, repair triggers and synthesizes new skills. Quantifying the resilience of this fallback under realistic perception noise is an important next step.

D. Reactivity and Update Frequencies

Across the experiments reported in this article, the symbolic strategy automaton evolves at the timescale of one symbolic transition, which corresponds to one online MICP solve—typically hundreds of milliseconds to a few seconds (Tables IV and V). Two complementary directions exist for tightening this loop in future work.

First, more frequent MICP updates—e.g., re-solving the online gait-fixed MICP at every NMPC tick rather than once per symbolic transition—are easily adaptable in the proposed framework: the delay-aware coordination scheme already supports asynchronous MICP solves of arbitrary duration, so any speed-up of the underlying mixed-integer solver, or any per-transition simplification, translates directly into a higher MICP update rate. The current limit is therefore the MICP solve time itself rather than any architectural constraint.

Second, more frequent symbolic-level planning is also achievable but requires careful design of the symbolic abstraction. Higher-frequency symbolic decisions would generally call for a finer abstraction—more atomic propositions per cell, finer terrain or robot-state granularity, or a larger skill repertoire. Reactive synthesis, however, scales exponentially in the number of variables in the specification bloem2012synthesis, so any finer abstraction must be balanced against synthesis cost. The high-level manager and symbolic repair adopted in this article already mitigate this scalability barrier; pushing toward truly real-time symbolic planning is an interesting future direction at the abstraction level rather than a limit of the framework itself.

E. Stronger Baseline Comparisons as Future Work

The empirical validation in this article compares against a pure-MIP planner and a nearest-feasible-foothold heuristic. Two stronger comparison points that we did not implement here remain valuable future targets: (i) optimization-based planners such as TOWR-style phase-based end-effector parameterization winkler2018gait, which jointly optimize gait timing and footholds in continuous time; and (ii) end-to-end perceptive reinforcement-learning controllers such as ANYmal-Parkour hoeller2023anymal and RLOC gangapurwala2022rloc. A fair comparison against either family requires either equipping them with a comparable symbolic decomposition layer or restricting our own framework to single-skill operation, which we did not pursue in the present validation.

F. Extension to Bipedal and Other Platforms

This article focuses on quadrupedal locomotion. The framework's symbolic-level machinery—terrain abstraction, GR(1) specifications, symbolic manager, symbolic repair, runtime

repair—is platform-agnostic and applies directly to bipedal robots. The MICP layer requires more careful adaptation: angular-momentum management becomes critical even at moderate speeds—a regime where the convex-dynamics simplification adopted here (Eq. (3)) becomes a stronger limitation. Generalizing to humanoids would also require revisiting the kinematic-feasibility re-targeting MICP to account for arm-swing and balance constraints. We see this as a natural future direction.

XI. Conclusion

In this work, we presented an integrated planning framework for terrain-adaptive locomotion that combines reactive synthesis with MICP, enabling safe and reactive responses in dynamically changing environments. To address unrealizable specifications arising from limited motion primitives, we introduced a symbolic repair approach that incorporates dynamic feasibility checks, automatically identifying missing transitions necessary for navigating adversarial terrains. In the online execution phase, we tackled the disparity between offline synthesis and real-world conditions by using an online MICP solver and a runtime repair process based on real-world terrain data, including unforeseen terrain conditions. Our approach not only enhances motion feasibility and safety but also provides a scalable solution for legged robots maneuvering in complex and safety-critical environments.

References